



Octane Security Report

Security Analysis of SailMoney: Protocol
6/24/2026

About Octane

Octane is a security intelligence platform purpose-built for modern software teams. We help engineering and security teams identify, investigate, and triage vulnerabilities faster—without disrupting the development process.

Combining automated static analysis with human-readable explanations, Octane bridges the gap between high-signal tooling and real-world developer workflows. Our platform provides contextualized vulnerability reports, clustered findings, and AI-enhanced guidance to make security reviews faster, clearer, and more actionable.

Octane was designed in collaboration with top auditors and engineers to meet the unique demands of web3 and next-gen software systems. We work closely with security researchers, protocol teams, and developer-first organizations to continuously improve our threat detection models and surface what actually matters.

Our clients include leading teams across DeFi, L2 protocols, infrastructure tooling, and security research. Octane integrates seamlessly with developer workflows—from GitHub automation to custom feedback tooling—so teams can reduce noise, eliminate risk, and ship with confidence.

Octane is backed by deep domain knowledge in static analysis, program synthesis, and exploit modeling, with capabilities designed for both technical depth and usability.

To learn more or request a demo, visit www.octane.security, or reach out to us at info@octane.security.

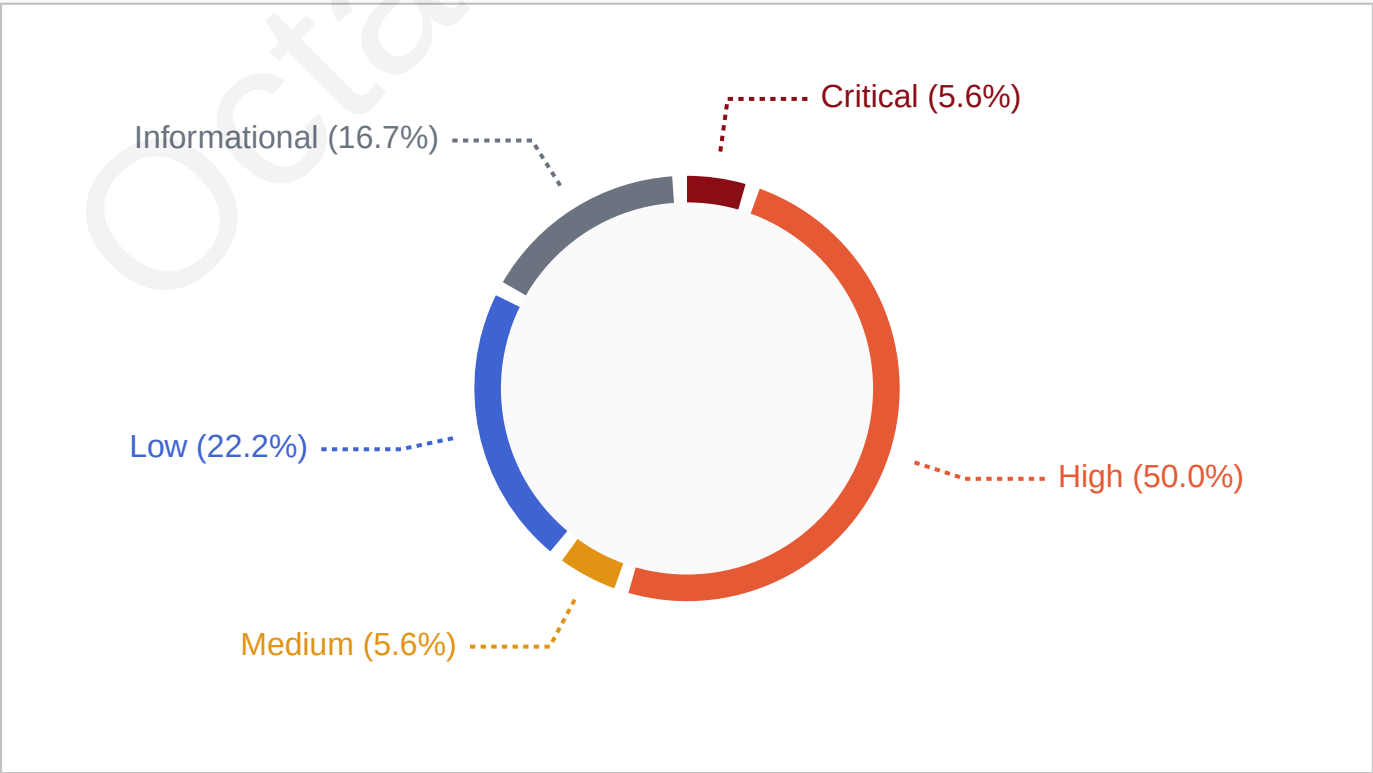
Octane Security
Distributed team with a global presence
www.octane.security

"Summary of findings"

This analysis reviewed the SailMoney: Protocol smart contracts using Octane’s automated analysis and included team feedback on findings.

The analysis identified a total of 18 issues. After triage and review, 18 of these issues have been resolved or acknowledged.

Vulnerabilities Confirmed	18/18
Critical	01/01
High	09/09
Medium	01/01
Low	04/04
Informational	03/03



Vulnerabilities & Warnings

Vulnerability #1

Missing `ctx.value` check in `SwapPermission` templates allows arbitrary ETH to be forwarded to routers enabling theft via `refundETH`

Severity: CRITICAL

Likelihood: HIGH

Impact: HIGH

Status: Resolved

Fixed in PR #52. `SwapPermission.evaluate` and `SwapPermissionNoOracle.evaluate` now reject any dispatch carrying native value (if `ctx.value != 0`) return false;) as the first check, so no ETH can be forwarded to a router and swept via `refundETH`. Matches the nonzero-`ctx.value` rejection the ERC-20 templates already enforced. Covered by regression tests; 21 swap-template duplicates resolved by the same guard in both templates.

`SwapPermission` and `SwapPermissionNoOracle` do not constrain the ETH value forwarded with dispatch calls. `SailKernel` forwards the signed value to the router, and `Uniswap V3 exactInputSingle` is payable. A malicious manager can attach large ETH to an otherwise-valid swap, causing the Safe to send ETH to the router. On canonical V3 routers, anyone can sweep this ETH via `refundETH()`, resulting in direct theft.

`SailKernel.dispatch` executes module calls with the provided value, and only the named permission is evaluated. In both `SwapPermission` and `SwapPermissionNoOracle`, `evaluate()` validates router/token allowlists, recipient pin, per-tx amountIn cap, and a price band (or sanity band), but never checks `ctx.value`. Therefore, a malicious manager can sign a dispatch for a payable `Uniswap V3 exactInputSingle` swap that otherwise satisfies all checks and attach arbitrary ETH. The Safe forwards this ETH to the router. Canonical `Uniswap V3` peripheral routers are payable on `exactInputSingle` and expose `refundETH()`, which transfers the router's ETH balance to `msg.sender` without authentication. The attacker (or any third party) can then call `refundETH()` to steal the forwarded ETH. Even when `tokenIn == WETH`, the router only wraps `amountIn`, leaving any surplus ETH on the router to be swept. Other ERC-20 templates in the repo explicitly reject nonzero `ctx.value`, highlighting this omission. The result is direct, material loss of principal funds from the Safe, with trivial exploitation in ordinary configurations that allowlist V3 routers.

file: contracts/templates/SwapPermission.sol

```
164     function evaluate(bytes calldata txData, Context calldata ctx)
        external view returns (bool) {
```

```
165   if (!isAllowedRouter[ctx.account][ctx.target]) return  
false;  
166   Slot storage s = _slots[ctx.account];
```

```
164   function evaluate(bytes calldata txData, Context calldata ctx)  
external view returns (bool) {  
165 + if (ctx.value != 0) return false;  
166   if (!isAllowedRouter[ctx.account][ctx.target]) return  
false;  
167   Slot storage s = _slots[ctx.account];
```

Severity

Impact Explanation: [High] The Safe's principal funds (ETH) are directly and materially lost: either actively stolen via refundETH on canonical Uniswap V3 routers, or permanently stranded/lost on non-refunding routers.

Likelihood Explanation: [High] Preconditions are ordinary for this template's intended use: allowlisting Uniswap V3 routers and holding ETH are common, and the manager may be adversarial by scope. Exploitation is trivial (single dispatch plus a simple router call) with clear profit incentive.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1: A Safe with SwapPermission registered allowlists a Uniswap V3 router and a token pair (e.g., USDC->WETH). The Safe holds ETH. A malicious manager submits dispatch to the V3 router with value=90 ETH and exactInputSingle data for a small amountIn ($\leq$ cap), recipient=account, and amountOutMinimum meeting the oracle band. The permission approves (no ctx.value check). The Safe sends 90 ETH to the router; only the small token swap executes. The attacker then calls refundETH() from their EOA and sweeps the router's ETH to themselves, stealing the 90 ETH.

Preconditions / Assumptions

- (a). A Safe account is registered and holds ETH
- (b). SwapPermission (or SwapPermissionNoOracle) is registered for the account
- (c). The permission configuration allowlists a Uniswap V3 router and compatible token pair
- (d). The manager key is adversarial or compromised (per scope)
- (e). Canonical V3 router behavior: exactInputSingle is payable; refundETH() sends ETH balance to msg.sender

Scenario 2.

Scenario 2: Same setup, but tokenIn=WETH and the Safe



holds ETH. The attacker dispatches to V3 `exactInputSingle` with `value=100 ETH`, `amountIn=0.1 WETH ( $\leq$  cap)`. The permission approves. The router wraps only 0.1 ETH into WETH and executes the swap; the 99.9 ETH surplus remains on the router. The attacker calls `refundETH()` and steals 99.9 ETH.

Preconditions / Assumptions

- (a). A Safe account is registered and holds ETH
- (b). `SwapPermission` (or `SwapPermissionNoOracle`) is registered for the account
- (c). The permission configuration allowlists a Uniswap V3 router and includes WETH as `tokenIn`
- (d). The manager key is adversarial or compromised (per scope)
- (e). Canonical V3 router behavior: `exactInputSingle` is payable; only `amountIn` is wrapped; `refundETH()` sends ETH balance to `msg.sender`

Scenario 3.

Scenario 3 (conditional): A payable router without a public refund path is allowlisted. The attacker overfunds an otherwise-valid swap. The permission approves; the Safe forwards the ETH; the surplus remains on the router but cannot be refunded. The Safe's ETH is effectively burned to the router (permanent loss) unless owners have another recovery path.

Preconditions / Assumptions

- (a). A Safe account is registered and holds ETH
- (b). `SwapPermission` (or `SwapPermissionNoOracle`) is registered for the account
- (c). The permission configuration allowlists a payable router lacking a public refund path
- (d). The manager key is adversarial or compromised (per scope)

Vulnerability #2

Non-atomic configure+register in MandateFactory attach/attachBatch/replace allows front-run activation of stale template config causing fund exfiltration

Severity: HIGH

Likelihood: MEDIUM

Impact: HIGH

Status: Resolved

Fixed in PR #59. Closed by epoch-binding: the kernel tracks a per-(account, permission) registrationEpoch pushed into Context, and each template denies unless configuredEpoch == ctx.configEpoch. A front-run register leaves the stale config's stamp at the old epoch, so the manager cannot dispatch under stale bounds; the victim re-enables with a standalone configure for the new epoch.

MandateFactory bundles template.configure with kernel register/replace but the two legs are independently authenticated and not cryptographically bound. SailKernel signatures bind only permission addresses (not config), and template config persists off-kernel across detach. A mempool observer can front-run only the kernel leg (register/replace), activating the permission immediately under stale per-account config while the victim's bundled tx later reverts and rolls back the fresh configure write. An adversarial manager can then dispatch calls under the stale, more-permissive rules to move funds out.

In MandateFactory.attach/attachBatch/replace, the factory first calls template.configure(account, params, ...), then calls SailKernel.registerPermission(s)/replacePermission with a separate EIP-712 signature. SailKernel's REGISTER_PERMISSION/REPLACE_PERMISSION type hashes bind only (account, permission) or (account, oldPermission, newPermission) with a signer nonce and deadline; they do not include a paramsHash or any binding to configuration state. The kernel register/replace functions are public and authenticate only the permissionSigner's signature; msg.sender is not restricted. Templates store per-account config in their own mappings and do not clear it on detach; detach only revokes at the kernel. Dispatch authorizes by permission address only; evaluate/evaluateBatch read template mapping state directly.

A mempool observer can copy the kernelSig from a pending MandateFactory bundle and front-run a direct call to SailKernel.registerPermission(s)/replacePermission. This immediately activates the permission under whatever stale config remains for that account in the template. When the victim's factory tx executes, the kernel leg now reverts with PermissionAlreadyRegistered (or equivalent in replace), which reverts the entire transaction and rolls back the fresh configure write. With the permission live under stale, permissive bounds and the manager treated as potentially adversarial within those bounds, the

manager can immediately dispatch authorized calls to move funds out (e.g., batch approve/swap-to-attacker/reset, or borrow-and-drain using cap-only stale rules).

file: contracts/templates/ConfigurablePermission.sol

```
39     bytes4 private constant ERC1271_MAGIC = 0x1626ba7e;  
40
```

```
41     bytes32 public constant CONFIGURE_TYPEHASH =  
42     keccak256("Configure(address account,bytes32  
paramsHash,uint256 nonce,uint256 deadline)");
```

```
39     bytes4 private constant ERC1271_MAGIC = 0x1626ba7e;  
40  
41     +// SECURITY-TODO: Add per-account config-hash binding for  
kernel registration.  
42     +// Implement: mapping(address => bytes32) _lastParamsHash;  
function configHash(address) external view returns (bytes32).  
43     +// Set _lastParamsHash[account] = keccak256(params) in both  
configure() and configureDirect().  
44     +  
45     bytes32 public constant CONFIGURE_TYPEHASH =  
46     keccak256("Configure(address account,bytes32  
paramsHash,uint256 nonce,uint256 deadline)");
```

Severity

Impact Explanation: [High] Once a stale, permissive permission is re-activated via front-run, the adversarial manager can immediately dispatch authorized calls that move assets from the Safe to attacker-controlled addresses (e.g., batch approve/swap-to-attacker/reset), resulting in direct, material loss of principal for the affected account.

Likelihood Explanation: [Medium] Exploitation requires uncommon but realistic conditions: the same template address must have stale permissive config for the account, the victim must use MandateFactory bundle(s) via public orderflow exposing a valid kernel signature, and the manager must be adversarial. These constraints are significant but do not depend on rare/exceptional states or integration failures, making the scenarios uncommon yet realistic.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (stale ApproveAndCallBatchPermission): Account A previously configured ApproveAndCallBatchPermission P with requireRecipientIsAccount=false, sufficient caps, and relevant tokens/spenders/pairs, then detached it (stale P._cfg[A] remains). The permissionSigner submits MandateFactory.attach(A, P, new stricter params with recipient pin, configureSig, kernelSig) in public mempool. An attacker front-runs SailKernel.registerPermission(A, P, kernelSig), paying the fee, so P is live under stale config. The adversarial manager M immediately dispatches a 3-call bracket [USDC.approve(router, amount); swapExactTokensForTokens(..., to=attackerEOA); USDC.approve(router, 0)], which passes because recipient isn't constrained under stale config. The victim's bundle later reverts on PermissionAlreadyRegistered, rolling back the configure write, and funds are exfiltrated to attackerEOA.

Preconditions / Assumptions

(a). The same shared template address (ApproveAndCallBatchPermission) was previously configured for account

- A with permissive settings (e.g., `requireRecipientIsAccount=false`, sufficient caps and allowlists).
- (b). The permission was detached from the kernel; stale per-account config remains in the template mapping.
 - (c). The account's manager M is adversarial within the bounds of registered permissions.
 - (d). `SailKernel` is not paused; `sessionActive` for the account is true.
 - (e). The `permissionSigner`'s kernel signature (`registerPermission`) is valid, unexpired, and visible in the public mempool.
 - (f). The attacker can front-run and pay the kernel registration fee.
 - (g). The victim submits the bundled `MandateFactory.attach` through public orderflow.

Scenario 2.

Scenario 2 (borrow-and-drain with two stale templates): Account A previously used `BorrowPermission B` in amount-cap-only mode (both oracles unset) and `ApproveAndCallBatchPermission P` with recipient unconstrained; both were detached, leaving stale configs. The `permissionSigner` submits `MandateFactory.attachBatch(A, [B, P], [new stricter params for both], kernelBatchSig)`. An attacker front-runs `SailKernel.registerPermissions(A, [B, P], kernelBatchSig)`, activating both under stale configs. The adversarial manager M borrows stablecoins using B (1tv check bypassed in cap-only mode) and then uses P's 3-call bracket to move the assets to `attackerEOA`. The factory bundle reverts on `PermissionAlreadyRegistered`, rolling back both configure writes.

Preconditions / Assumptions

- (a). `BorrowPermission` and `ApproveAndCallBatchPermission` each retain permissive stale configs for the same account A (`BorrowPermission` in amount-cap-only mode; `ApproveAndCallBatchPermission` with recipient unconstrained).
- (b). Both permissions were detached from the kernel; stale per-account config remains.
- (c). The account's manager M is adversarial within the bounds of registered permissions.
- (d). `SailKernel` is not paused; `sessionActive` for the account is true.
- (e). The `permissionSigner`'s batch kernel signature (`registerPermissions`) is valid, unexpired, and visible in the public mempool.
- (f). The attacker can front-run and pay the kernel registration fees.



(g). The victim submits the bundled `MandateFactory.attachBatch` through `public orderflow`.

Scenario 3.

Scenario 3 (replace to previously used shared address with stale config): Account A uses `MandateFactory.replace(A, O, N, new stricter params for N, configureSig, kernelReplaceSig)` to tighten rules, but N is a shared template address previously used by A, retaining permissive stale config. An attacker front-runs `SailKernel.replacePermission(A, O, N, kernelReplaceSig)`, atomically removing O and registering N under stale config. The adversarial manager M immediately exfiltrates using N (e.g., an unconstrained recipient in a batch bracket). The victim's replace later reverts on `PermissionAlreadyRegistered`, rolling back the configure write and leaving N live under stale rules.

Preconditions / Assumptions

- (a). `replacePermission` targets a permission address N that was previously configured for account A and retains permissive stale config.
- (b). The victim submits `MandateFactory.replace` in public mempool with a valid, unexpired `kernelReplaceSig`.
- (c). The attacker can front-run `SailKernel.replacePermission`, paying the fee.
- (d). `SailKernel` is not paused; `sessionActive` for the account is true.
- (e). The account's manager M is adversarial within the bounds of registered permissions.



Vulnerability #3

Non-rotating manager/batch nonce epochs on session reactivation in `SailKernel.activateSession` causes execution of attacker pre-signed dispatches after reactivation

Severity: HIGH

Likelihood: MEDIUM

Impact: HIGH

Status: Resolved

Fixed in PR #56. `activateSession` now rotates the manager and batch nonce epochs (mirroring `revokeSession`), so a dispatch pre-signed during suspension cannot execute on reactivation.

`SailKernel.revokeSession()` bumps manager/batch nonce epochs and disables dispatch, but `SailKernel.activateSession()` only flips `sessionActive` back to true and does not rotate nonce epochs. A manager (treated as potentially adversarial) can pre-sign dispatch/dispatchBatch messages during suspension (post-revoke epoch). These signatures cannot execute while suspended but become valid immediately upon activation, enabling token drains or risky actions if permissions allow.

`SailKernel` enforces dispatch authorization via manager EIP-712 signatures keyed by per-account nonces: `managerNonces` for dispatch and `batchNonces` for dispatchBatch. When the permissionSigner calls `revokeSession(account,...)`, the kernel sets `configs[account].sessionActive=false` and bumps both `managerNonces[account]` and `batchNonces[account]` by a large `NONCE_EPOCH_INCREMENT`, invalidating all manager signatures from prior epochs. However, `activateSession(account,...)` only sets `configs[account].sessionActive=true` and consumes a signer nonce; it does not rotate manager/batch nonces. As a result, any manager-signed messages created after the `revokeSession` epoch bump (i.e., while `sessionActive=false`) remain valid and executable as soon as the session is reactivated, provided deadlines have not expired, the permission remains registered, and the protocol is not paused. This creates an operationally surprising window: a compromised or adversarial manager can pre-sign valid calls during downtime and execute them immediately upon reactivation, potentially in the same block, leading to direct token loss (e.g., via batch patterns that allow external recipients) or creating immediate position risk (e.g., rapid borrows). The kernel's comments tie nonce-epoch bumps to restriction operations (`revoke/replace/revokeSession`) and not to activation, indicating this behavior is by design, but it nonetheless exposes a post-activation execution window that operators may not expect to invalidate automatically without an additional epoch-bumping step (e.g., second revoke while inactive, revoke/replace permissions, or manager rotation).

file: contracts/core/SailKernel.sol

```
953     signerNonces[account] = nonce + 1;
954     configs[account].sessionActive = true;

955     emit SessionActivated(account);
956 }
```

```
953     signerNonces[account] = nonce + 1;
954     configs[account].sessionActive = true;
955 +// Rotate nonce epochs on activation to invalidate any
    manager-signed messages created during suspension.
956 + managerNonces[account] += NONCE_EPOCH_INCREMENT;
957 + batchNonces[account]   += NONCE_EPOCH_INCREMENT;
958     emit SessionActivated(account);
959 }
```


Severity

Impact Explanation: [High] Scenarios enable direct, material loss of principal funds (e.g., token drains) or create immediate position risk that can lead to liquidation and principal loss.

Likelihood Explanation: [Medium] Exploitation requires an adversarial or temporarily compromised manager during suspension, operator reactivation without an additional epoch-bumping step, valid registered permissions, and non-expired deadlines. These conditions are uncommon but realistic in operations and provide clear attacker incentives.

Exploitation

Exploitation Scenarios:

Scenario 1.

Token drain via `ApproveAndCallBatchPermission` with unconstrained recipient: While `sessionActive=false` (after `revokeSession` bumped epochs), the attacker holding or having held the manager key pre-signs a valid 3-call batch (`approve/spend/reset`) where the consuming swap sends output to the attacker's EOA. Upon `activateSession()`, because nonce epochs were not rotated, the pre-signed batch immediately verifies and executes, draining tokens from the Safe to the attacker.

Preconditions / Assumptions

- (a). Account registered with `SailKernel` and has `ApproveAndCallBatchPermission` registered
- (b). Permission configured with token/spender/consuming pair allowlists and `requireRecipientIsAccount=false`
- (c). `permissionSigner` previously called `revokeSession(account,...)` so `sessionActive=false` and nonce epochs advanced
- (d). Attacker holds or held the manager key during suspension and pre-signed a valid batch using current batch-Noncesaccount. `activateSession(account,...)` called by `permissionSigner` without any intervening nonce-epoch-bumping operation



(f). Deadlines on the attacker's signatures have not expired and governance is not paused when executing

Scenario 2.

BorrowPermission pre-signed borrows creating liquidation risk: During suspension (post-epoch bump), the attacker pre-signs multiple borrow dispatches that satisfy BorrowPermission (caps, allowlists, and if configured, LTV checks at execution). After activateSession(), these execute in quick succession, rapidly increasing the account's debt and pushing health factors toward liquidation, risking principal loss if markets move or if additional permissions move assets out.

Preconditions / Assumptions

- (a). Account registered with SailKernel and has BorrowPermission registered
- (b). BorrowPermission configured to allow the targeted protocols/assets with sufficient per-tx caps; oracles optional
- (c). permissionSigner previously called revokeSession(account,...) so sessionActive=false and nonce epochs advanced
- (d). Attacker holds or held the manager key during suspension and pre-signed valid borrow dispatches using current managerNoncesaccount. activateSession(account,...) called by permissionSigner without any intervening nonce-epoch-bumping operation
- (f). Deadlines valid and governance not paused at execution; external market conditions can move against the account to trigger liquidation risk

Scenario 3.

Same-block race at reactivation: The attacker monitors mempool and, when activateSession() is broadcast, submits pre-signed dispatch/dispatchBatch transactions to be ordered immediately after activation in the same block. Since activation doesn't rotate nonce epochs, the batched calls verify and execute before the operator can perform any nonce-rotating mitigation, enabling immediate fund loss under permissive permissions.

Preconditions / Assumptions

- (a). Any move-capable permission (e.g., ApproveAndCallBatchPermission) registered and configured to allow the intended calls
- (b). permissionSigner previously called revokeSession(account,...) so sessionActive=false and nonce epochs advanced



- (c). Attacker holds or held the manager key during suspension and pre-signed valid dispatches/batches using current nonces
- (d). permissionSigner submits activateSession(account,...) and a builder/MEV actor orders attacker's transactions immediately after in the same block
- (e). No intervening epoch-bumping operation between revoke and activation; deadlines valid; governance not paused

Octane Security

Vulnerability #4

Insufficient authorization in SailKernel.registerAccount during Safe.setup allows attacker-chosen principals causing fund theft

Severity: HIGH

Likelihood: MEDIUM

Impact: HIGH

Status: Resolved

Fixed in PR #53. registerAccount now requires a Safe-owner EIP-712 signature verified via Safe-core checkSignatures, closing the setup-window registration; the prior nonce-only guard was forgeable via setup-delegatecall sstore.

SailKernel.registerAccount accepts self-registration from a Safe based only on proxy codehash and the kernel module being enabled. Under Safe v1.4.1, a setup delegatecall helper can first enable the module via a self-call and then immediately call registerAccount from the Safe, setting attacker-controlled permissionSigner and manager without any owner-approved transaction.

SailKernel.registerAccount relies on two checks: (1) the caller's runtime codehash must match a trusted Safe proxy codehash, and (2) the caller Safe must already have this kernel enabled as a module. It does not require any post-setup owner-approved execTransaction nor that Safe.nonce() > 0. In Gnosis Safe v1.4.1, Safe.setup performs a delegatecall to a user-supplied target. Code executed under this delegatecall runs with address(this) == the Safe, allowing a helper to: (a) self-call enableModule(kernel) to satisfy the onlySelf guard, and (b) immediately make an external CALL to SailKernel.registerAccount from the Safe during setup. At that moment, both kernel checks pass (genuine Safe proxy codehash and module enabled). The result is an unauthorized registration that sets attacker-controlled permissionSigner and manager on the account. After the Safe is later funded, the attacker, acting as permissionSigner, can register a permissive permission, and as manager, can dispatch calls via the kernel module to transfer ETH/tokens out of the Safe using Safe.execTransactionFromModule. Owners can later remediate by rotating manager or disabling the module, but this does not prevent immediate theft after funding.

file: contracts/core/SailKernel.sol

```
42     function isModuleEnabled(address module) external view returns
    (bool);

43 }
```

```

...

458     error ModuleNotEnabled();
459

460     /// @dev Thrown by `createAccount` when the provided Safe
factory is not in governance's trusted allowlist.
461     error UntrustedFactory(address factory);

...

656     if (!governance.trustedSafeProxyCodehash(codehash)) revert
UntrustedProxyCodehash(codehash);
657

658     if (!ISafe(msg.sender).isModuleEnabled(address(this)))
revert ModuleNotEnabled();

```

```

42     function isModuleEnabled(address module) external view returns
(bool);
43 + function nonce() external view returns (uint256);
44 }

...

459     error ModuleNotEnabled();
460
461 + error SetupNotFinalized();
462 +
463     /// @dev Thrown by `createAccount` when the provided Safe
factory is not in governance's trusted allowlist.
464     error UntrustedFactory(address factory);

...

659     if (!governance.trustedSafeProxyCodehash(codehash)) revert
UntrustedProxyCodehash(codehash);
660
661 + if (ISafe(msg.sender).nonce() == 0) revert SetupNotFinal-
ized();
662 +
663     if (!ISafe(msg.sender).isModuleEnabled(address(this)))
revert ModuleNotEnabled();

```

Octane Security



Severity

Impact Explanation: [High] Once attacker-controlled permissionSigner and manager are set, the attacker can register a permissive permission and dispatch arbitrary fund-moving calls via the kernel module, leading to direct, material loss of user funds.

Likelihood Explanation: [Medium] Exploitation requires users to onboard through a malicious or compromised setup helper (a realistic external constraint) or to adopt/fund a pre-created Safe (less common). No special on-chain states or complex synchronization are required once the helper is used.

Exploitation

Exploitation Scenarios:

Scenario 1.

Malicious onboarding helper: A user creates a Safe through a dapp that supplies `Safe.setup(to,data)`. The helper, executed via `delegatecall` during setup, first self-calls `enableModule(kernel)` and then calls `SailKernel.registerAccount` with attacker-chosen `permissionSigner` and `manager`. After the user funds the Safe, the attacker registers a permissive permission and drains ETH/tokens via `SailKernel.dispatch` through `Safe.execTransactionFromModule`.

Preconditions / Assumptions

- (a). `SailKernel` is deployed and accessible.
- (b). Governance has allowlisted the canonical `SafeProxy` runtime codehash (normal configuration).
- (c). User uses an onboarding flow that supplies `Safe.setup(to,data)` with a malicious or compromised helper.
- (d). User funds the newly created Safe.

Scenario 2.

Pre-created Safe adoption: An attacker deploys a Safe with the victims as owners, using a setup helper to enable the kernel module and call `SailKernel.registerAccount` with attacker-chosen `permissionSigner` and `manager` during setup. If the victims later fund or adopt that Safe address,



the attacker registers a permissive permission and drains funds via `SailKernel.dispatch`.

Preconditions / Assumptions

- (a). `SailKernel` is deployed and accessible.
- (b). Governance has allowlisted the canonical `SafeProxy` runtime codehash (normal configuration).
- (c). Attacker deploys a `Safe` proxy with the intended victims as owners and uses a setup helper to enable the kernel module and call `registerAccount` during setup.
- (d). Victims later fund or adopt the attacker-created `Safe` address.

Vulnerability #5

Fee-asset change without policy-state migration in `SailKernel.setFeePolicy` + `StandardFeePolicy` causes one-time inflated performance fees and loss of principal

Severity: HIGH

Likelihood: MEDIUM

Impact: HIGH

Status: Resolved

Fixed across two PRs. PR #57 closed the asset-denomination sub-part: `setFeePolicy` pins each (account, policy) pair to its fee asset via an explicit bound flag, reverting `FeePolicyAssetMismatch` on a denomination swap of a reused instance (to change asset, point at a fresh policy instance). PR #65 closed the policy-state-migration sub-part: the new `IFeePolicy.onAttach` lifecycle hook re-anchors per-account fee state on (re)attach (resets `lastCollectionTimestamp` and re-bases the high-water mark via `pendingReanchor`), so detaching and reattaching the same policy no longer over-collects management fees for the dormant interval or charges performance fees on gains booked while detached.

Changing an account's fee asset via `SailKernel.setFeePolicy` while keeping the same `StandardFeePolicy` instance leaves the policy's per-account high-water mark (HWM) in the old units. On the next `collectFees`, `StandardFeePolicy.computeFee` compares `currentNav` in the new units against HWM in the old units, inflating (or suppressing) performance fees. The kernel enforces only $\text{grossFee} \leq \text{maxFee}$, enabling a one-time over-collection of principal from the Safe.

`SailKernel.setFeePolicy` updates `configs[account].feeAsset` and `configs[account].feePolicy` (allowing the same policy address to remain) but does not coordinate any state migration with the fee policy. `StandardFeePolicy` keeps per-account state (`highWaterMark`, `lastCollectionTimestamp`, `appliedManagementFeeBps`, `appliedPerformanceFeeBps`) and assumes unit/denomination continuity across time. Its `computeFee(account, currentNav)` calculates performance fee as $\max(\text{currentNav} - \text{highWaterMark}, 0) \times \text{performanceFeeBps} / 10,000$. After a fee-asset switch, `currentNav` is in the new token units while `highWaterMark` remains in the old units, creating an artificial profit (or suppressing true profit) on the first post-change collection. `SailKernel.collectFees` validates only that `feeToken` matches the new `feeAsset` and that $\text{grossFee} \leq \text{IFeePolicy.computeFee}(\text{account}, \text{currentNav})$, then calls `IFeePolicy.recordCollection` and executes transfers. `recordCollection` subsequently sets $\text{HWM} = \max(\text{HWM}, \text{currentNav})$, restoring unit consistency for future calls, but it cannot undo the initial over-collection. No NAV fraud is required; `currentNav` can be honest in the new units. Preconditions include that the account is already initialized in `StandardFeePolicy` (`hwmSeeded true`, `lastCollectionTimestamp` set), that at least `MIN_COLLECTION_INTERVAL` (1 day) has elapsed since the last collection,

and that the Safe holds a balance in the new fee asset. This produces a one-time opportunity for the manager to collect up to the (inflated) policy ceiling from the Safe, causing direct loss of principal. The kernel provides only documentation warnings about denomination consistency and no on-chain invariant or migration hook to prevent this.

file: contracts/core/SailKernel.sol

```
244  //
    -----

245  // Principal tracking
246  //
    -----

...

509  ///      in governance's trusted allowlist. Prevents
      upgradeable/metamorphic policies.
510  error UntrustedFeePolicy(address policy);

511
512  /// @dev Thrown by `replacePermissions` when `oldPermissions`
      and `newPermissions` have different lengths.

...

977  );
978  signerNonces[account] = nonce + 1;

979  configs[account].feePolicy = newFeePolicy;
980  configs[account].feeAsset  = (newFeePolicy == address(0))
      ? address(0) : feeAsset;
```

244

```

//
-----
245 +/// @dev Per-account binding of a fee policy instance to a
    single fee asset to prevent cross-asset reuse.
246 + mapping(address account => mapping(address policy => address
    asset)) private _policyAssetBinding;
247 // Principal tracking
248 //
-----

...

511 ///      in governance's trusted allowlist. Prevents
    upgradeable/metamorphic policies.
512     error UntrustedFeePolicy(address policy);
513 +/// @dev Thrown by `setFeePolicy` when attempting to reuse a
    policy instance with a different fee asset.
514 + error FeePolicyAssetMismatch(address policy, address expected,
    address provided);
515
516 /// @dev Thrown by `replacePermissions` when `oldPermissions`
    and `newPermissions` have different lengths.

...

981 );
982     signerNonces[account] = nonce + 1;
983 +// Bind the currently configured policy to its fee asset
    if first seen for this account.
984 + address cp = configs[account].feePolicy;
985 + if (cp != address(0) && _policyAssetBinding[account][cp]
    == address(0))
986 + _policyAssetBinding[account][cp] = configs[account].fee-
    Asset;
987 +// Enforce that a policy instance is reused only with its
    originally bound asset for this account.
988 + if (newFeePolicy != address(0)) {
989 + address bound = _policyAssetBinding[ac-
    count][newFeePolicy]; if (bound == address(0)) _policyAssetBind-
    ing[account][newFeePolicy] = feeAsset; else if (bound != feeAsset)
    revert FeePolicyAssetMismatch(newFeePolicy, bound, feeAsset);
990 +}
991     configs[account].feePolicy = newFeePolicy;
992     configs[account].feeAsset = (newFeePolicy == address(0))
    ? address(0) : feeAsset;

```

Severity

Impact Explanation: [High] Direct, material loss of principal funds from the Safe on the first post-change collection, as collectFees transfers tokens/ETH out to recipients based on an inflated performanceFee ceiling.

Likelihood Explanation: [Medium] Exploitation requires realistic but non-trivial preconditions: a fee-asset change while reusing the same StandardFeePolicy instance without migration, at least 1 day since last collection, and sufficient balance in the new asset. These are uncommon but plausible operational states across managed accounts.

Exploitation

Exploitation Scenarios:

Scenario 1.

USD stable (6 decimals) → WETH: The permissionSigner switches the account's fee asset to WETH while keeping the same StandardFeePolicy. StandardFeePolicy.highWaterMark remains in old USD units. The manager calls collectFees with feeToken=WETH and an honest currentNav in wei. computeFee treats currentNav - HWM as a huge positive number, allowing a large performanceFee. The manager sets grossFee ≤ maxFee and drains WETH from the Safe on this first post-change collection.

Preconditions / Assumptions

- (a). Account registered with SailKernel and using StandardFeePolicy (allowlisted).
- (b). StandardFeePolicy initialized for the account: hwm-Seeded[account] == true and lastCollectionTimestamp[account] != 0.
- (c). At least MIN_COLLECTION_INTERVAL (1 day) has elapsed since the last recordCollection.
- (d). PermissionSigner submits setFeePolicy(account, same StandardFeePolicy address, feeAsset=WETH) successfully (session active, not paused for non-zero policy).
- (e). The Safe holds a material WETH balance.
- (f). Manager session active; caller is manager or account; feeToken in collectFees is WETH; NAV provided honestly in



WETH units.

Scenario 2.

WETH (18 decimals) → USD stable (18 decimals): The permissionSigner switches fee asset to a USD-denominated 18-decimal token while reusing the same StandardFeePolicy. The stored HWM is in ETH units, but currentNav is now in USD units (18 decimals). computeFee sees a large positive difference and authorizes a large performanceFee in the USD token. The manager collects up to maxFee, transferring USD tokens from the Safe.

Preconditions / Assumptions

- (a). Account registered with SailKernel and using StandardFeePolicy (allowlisted).
- (b). StandardFeePolicy initialized for the account: hwm-Seeded[account] == true and lastCollectionTimestamp[account] != 0.
- (c). At least MIN_COLLECTION_INTERVAL (1 day) has elapsed since the last recordCollection.
- (d). PermissionSigner submits setFeePolicy(account, same StandardFeePolicy address, feeAsset=USD18 token).
- (e). The Safe holds a material balance of the USD 18-decimal token.
- (f). Manager session active; caller is manager or account; feeToken in collectFees is the USD18 token; NAV provided honestly in USD18 units.

Scenario 3.

Switch to native ETH: The permissionSigner sets feeAsset=address(0) (ETH) on the same StandardFeePolicy. The HWM is in the prior token's units, but currentNav is now in wei. computeFee inflates performanceFee; collectFees pays out via _safeTransferETH to treasury/distributor/feeManager, causing direct ETH loss from the Safe on the first post-change collection.

Preconditions / Assumptions

- (a). Account registered with SailKernel and using StandardFeePolicy (allowlisted).
- (b). StandardFeePolicy initialized for the account: hwm-Seeded[account] == true and lastCollectionTimestamp[account] != 0.
- (c). At least MIN_COLLECTION_INTERVAL (1 day) has elapsed since the last recordCollection.
- (d). PermissionSigner submits setFeePolicy(account, same StandardFeePolicy address, feeAsset=address(0) (ETH)).
- (e). The Safe holds a material ETH balance and the feeRecipient (feeManager) is payable.



(f). Manager session active; caller is manager or account; feeToken in collectFees is address(0); NAV provided honestly in wei.

Octane Security

Vulnerability #6

Integer truncation and floor rounding in BorrowPermission _ltvCheck allows per-step LTV ceiling bypass leading to potential fund exfiltration

Severity: HIGH

Likelihood: MEDIUM

Impact: HIGH

Status: Resolved

Fixed in PR #54. Replaced the silently-defeated LTV check with a fail-closed amount-based ceiling that correctly applies the collateral and borrow oracle decimals; regression tests added.

BorrowPermission computes the per-call LTV using an intermediate integer-quantized borrow value (borrowScaled) and final floor division, which undercounts the borrow's true numeraire value. Any per-call borrow worth <1 numeraire unit yields borrowScaled=0 and passes any LTV ceiling; near-boundary borrows slightly above the ceiling can also pass due to rounding down. When a movement permission is also registered (e.g., ApproveAndCallBatchPermission with unconstrained recipient), an adversarial manager can borrow and exfiltrate assets, leaving the Safe with debt.

The IOOracle spec defines price as per-base-unit value: 1 base unit = price/ 10^{decimals} numeraire units. In BorrowPermission._ltvCheck, the code first computes borrowScaled = Math.mulDiv(amount, borPrice, 10^{borDec}), which floors the real borrow value to an integer count of whole numeraire units (discarding up to almost 1 full unit). It then computes ltvBps = Math.mulDiv(borrowScaled * 10_000, 10^{colDec} , colValue), again flooring. Consequences: (1) Any single borrow with true numeraire value <1 produces borrowScaled=0, so ltvBps=0 and the call bypasses any maxLtvBps. (2) A slightly-over-threshold borrow can pass because rounding down undercounts the borrow. This flaw does not rely on malicious oracles; it follows directly from the documented oracle semantics and the permission's arithmetic. With a commonly used batch approve/consume/reset permission that leaves the recipient unconstrained, an adversarial manager can borrow and immediately route the tokens to their own address, leaving the Safe with the debt. Even without a movement permission, the per-step LTV ceiling is defeated for sub-1-unit borrows, enabling riskier position building than intended.

file: contracts/templates/BorrowPermission.sol

```
214     if (borPrice == 0) return false;
```

```

215
216 -// Normalise the borrow side to a value quantity:
    borrowScaled = amount * borPrice / 10^borDec.
217 - uint256 borrowScaled = Math.mulDiv(amount, borPrice, 10
    ** uint256(borDec));
218 -// LTV = borrowScaled / (colValue / 10^colDec). Do NOT
    pre-divide colValue by 10^colDec:
219 -// that integer division floors away the fractional
    collateral and overstates the LTV,
220 -// wrongly blocking borrows that are actually within the
    ceiling. Instead fold 10^colDec into
221 -// the numerator – mulDiv carries the (borrowS-
    caled*10_000)*10^colDec product at full
222 -// precision in a 512-bit intermediate – and divide by
    the full-precision colValue exactly
223 -// once. colValue == 0 is already guarded above, so the
    division is safe.
224 - uint256 ltvBps = Math.mulDiv(borrowScaled * 10_000, 10 **
    uint256(colDec), colValue);
225 - return ltvBps <= s.maxLtvBps;
226 }

```

```

214  if (borPrice == 0) return false;
215
216 +// Enforce: amount * borPrice / 10^borDec <=
    (maxLtvBps/10_000) * (colValue / 10^colDec)
217 +// Compute a conservative max-allowed amount in base units
    (fail-closed via floors).
218 + uint256 scaledCollateral = Math.mulDiv(colValue, 10 **
    uint256(borDec), 10 ** uint256(colDec));
219 + uint256 scaledBound = Math.mulDiv(scaledCollateral,
    s.maxLtvBps, 10_000);
220 + uint256 maxAmountAllowed = scaledBound / borPrice;
221 + return amount <= maxAmountAllowed;

222 }

```

Severity

Impact Explanation: [High] In the most severe (but plausible) configuration, the flaw enables direct, material loss of principal funds: borrowed tokens are immediately exfiltrated via an allowed batch approve/consume/reset flow, leaving the Safe with debt.

Likelihood Explanation: [Medium] The core LTV bypass requires no special constraints and is trivial to trigger; however, direct theft in Scenario 1 additionally depends on the presence and configuration of a movement permission with unconstrained recipient, which is plausible but not universal.

Exploitation

Exploitation Scenarios:

Scenario 1.

Sub-1-numeraire borrow bypasses LTV and is exfiltrated:

The manager borrows an allowed asset whose per-token price is 0, and appropriate allowlists for the lending protocol and asset

(c). Oracles are honest and fresh per docs: borrow oracle returns per-base-unit price in a common numeraire (borDec in 0..77), collateral oracle returns aggregate collateral (colDec in 0..77)

(d). Underlying lending protocol accepts the requested borrow amount

(e). The asset's per-token price is 0), and with appropriate protocol/asset allowlists

(c). Oracles are honest and fresh per docs (per-base-unit price convention; consistent numeraire)

(d). Underlying lending protocol accepts small borrow amounts on the chosen reserve

(e). Manager selects amounts per call so each true borrow value is 0

(d). Manager chooses an amount that makes the true per-step LTV slightly above maxLtvBps (e.g., \$500.1 borrow on \$1000 collateral with a 50% ceiling)



Octane Security



Vulnerability #7

Missing binding between approved token/spender and consuming call in ApproveAndCallBatchPermission causes direct theft via stale allowances

Severity: HIGH

Likelihood: MEDIUM

Impact: HIGH

Status: Resolved

Fixed in PR #55. `evaluateBatch` now binds the consuming call's spender to the approved spender and its consumed asset to the approved token (7 decodable standard-ABI selectors; non-decodable fail-closed), plus a pre-batch zero-allowance check.

`ApproveAndCallBatchPermission` enforces an approve/consume/reset shape but never proves that the middle call actually spends the allowance granted in the first call. It only checks the consuming (target,selector) pair, optionally the first 32-byte word, and optionally a recipient, without binding the consumed token or spender to the approved pair. If the Safe has a pre-existing allowance on a different token to the consuming target (or its puller), the manager can spend that token while bracketing and resetting an unrelated allowance, enabling direct theft or unintended asset movements.

The template validates a strict 3-call shape: (0) `approve(spender, amount)` on an allowlisted token within a per-token cap; (1) a consuming call on an allowlisted (target, selector) pair; (2) `approve(spender, 0)` reset for the same token/spender. In `evaluateBatch`, `call[1]` only verifies that (target,selector) is allowlisted, `c1.value == 0`, and optionally that the first `uint256` word equals `approveAmount` (`requireAmountMatch`) and/or that a decodable recipient equals the account (`requireRecipientIsAccount`). Critically, it never enforces that `c1.target` equals the spender from `call[0]` or that the consumed asset in `c1` (e.g., Uniswap V2 `path[0]`, Uniswap V3 `tokenIn`, Aave asset) equals the token approved in `call[0]`. As a result, if the Safe carries a stale non-zero allowance on another token to the consuming target or an internal puller (e.g., `AllowanceTarget/Permit2`), the consuming call can pull that other token while the template only brackets and resets an unrelated allowance. `requireAmountMatch` does not fix this because it compares only the first 32-byte word (often not the amount for many selectors), and `requireRecipientIsAccount` only pins the recipient for a narrow decodable set without binding the consumed token/spender. `call[2]` resets only the bracketed token/spender pair, leaving the stale allowance used by `call[1]` unchanged. This enables direct, repeatable theft (when the recipient is unconstrained) or unauthorized asset conversions within the account.

```
309     if (!isConsumingPair[account][_pairKey(c1.target, sel)])  
        return false;  
310
```

```
311     if (_cfg[account].requireAmountMatch) {  
312         uint256 consumedAmount = _decodeFirstUint256(c1.data);
```

```

309  if (!isConsumingPair[account][_pairKey(c1.target, sel)])
    return false;
310
311  +// SECURITY FIX REQUIRED:
312  +// - Enforce c1.target == `spender` (from call[0]) to bind
    the consuming call to the approved spender.
313  +// - Decode the consumed asset per selector (e.g., V2
    path[0], V3 exactInputSingle.tokenIn,
314  +//   Aave deposit/supply `asset`, ERC-4626 `asset()` via
    staticcall) and require it equals `token`
315  +//   (from call[0]). Deny if the selector cannot be safely
    decoded.
316  +// - When `requireAmountMatch` is true, decode the
    selector-specific amount field (not the generic
317  +//   first word) and require it equals `approveAmount`.
318  +// - Deny opaque/aggregator/universal-router payloads
    unless an explicit expected puller is
319  +//   configured and verified. Optional hardening: require
    IERC20(token).allowance(account, spender)
320  +//   == 0 before call[0] to eliminate residual allowance
    on the same (token,spender) pair.
321  +
322  if (_cfg[account].requireAmountMatch) {
323  uint256 consumedAmount = _decodeFirstUint256(c1.data);

```

Severity

Impact Explanation: [High] In all valid scenarios, principal funds are directly and materially lost: tokens are pulled via stale allowances and sent to the attacker or otherwise converted against the user's interests.

Likelihood Explanation: [Medium] Exploitation requires specific but realistic preconditions (stale allowances to the consuming target or its puller, and permissive/default template configuration). These are common in practice but not universal.

Exploitation

Exploitation Scenarios:

Scenario 1.

Uniswap V2 direct theft with stale allowance: The manager submits a batch where `call[0]` approves token A to router R for a capped amount; `call[1]` calls `R.swapExactTokensForTokens` with `path=[B->X]`, `amountIn` set to the cap (if `amount-match` is on), `minOut=0`, and `to=attacker`; `call[2]` resets `A->R` to zero. Because the Safe has a stale `B->R` allowance, R pulls token B using that allowance and sends outputs to the attacker. The template never binds the consumed token (B) to the approved token (A) nor requires `cl.target==spender`.

Preconditions / Assumptions

- (a). The account is registered with `SailKernel` and `ApproveAndCallBatchPermission` is registered
- (b). `consumingPairs` includes (Uniswap V2 router R, `swapExactTokensForTokens`)
- (c). `tokens[]` includes token A with a positive cap; `spenders[]` includes R
- (d). The Safe has a non-zero stale allowance `B->R` and holds token B
- (e). `requireRecipientIsAccount` is false (default)
- (f). `requireAmountMatch` is either off (unbounded per batch) or on (`amountIn` equals `approveAmount`)

Scenario 2.



Aave V2/V3 direct theft via onBehalfOf=attacker: The manager submits a batch where call[0] approves token A to pool P; call[1] calls P.deposit/supply with asset=B, amount large, onBehalfOf=attacker; call[2] resets A->P to zero. With a stale B->P allowance, P pulls token B and mints aTokens to the attacker. requireAmountMatch must be off (first word is an address), and the template never binds asset B to the approved token A.

Preconditions / Assumptions

- (a). The account is registered with SailKernel and Approve-AndCallBatchPermission is registered
- (b). consumingPairs includes (Aave pool P, deposit/supply selector)
- (c). tokens[] includes token A with a positive cap; spenders[] includes P
- (d). The Safe has a non-zero stale allowance B->P and holds token B
- (e). requireRecipientIsAccount is false (default)
- (f). requireAmountMatch is false (first word is an address for these selectors)

Scenario 3.

Aggregator theft via AllowanceTarget/Permit2: The manager submits a batch where call[0] approves token A to an allowlisted spender S; call[1] calls an allowlisted aggregator with calldata instructing a swap of token B with recipient=attacker; call[2] resets A->S to zero. The aggregator's internal AllowanceTarget/Permit2 T pulls B using a stale B->T allowance. The template checks only (aggregator,selector) and optional recipient; it does not bind the real pulling spender or token to call[0].

Preconditions / Assumptions

- (a). The account is registered with SailKernel and Approve-AndCallBatchPermission is registered
- (b). consumingPairs includes (Aggregator, supported selector)
- (c). tokens[] includes token A with a positive cap; spenders[] includes an allowlisted spender (may differ from aggregator's puller)
- (d). The Safe has a non-zero stale allowance B->T where T is the aggregator's AllowanceTarget/Permit2, and holds token B
- (e). requireRecipientIsAccount is false (default)
- (f). requireAmountMatch is false (aggregator calldata often uses structs/bytes where the first word is not amount)

Vulnerability #8

Missing epoch binding between kernel revoke/replace and template configNonces in ConfigurablePermission enables stale-config replay at re-enable, allowing unauthorized fund movements

Severity: HIGH

Likelihood: MEDIUM

Impact: HIGH

Status: Resolved

Fixed in PR #59. registrationEpoch is bumped on the leaving side (revoke/replace/clear); the configure and identity signatures are epoch-bound and the template EIP-712 domain version bumped "1"→"2", so a stale-but-unexpired configure signature is rejected after a revoke/re-register cycle. Closes #2 by the same mechanism. Migration note: the domain bump invalidates all outstanding configure/identity signatures at deploy.

ConfigurablePermission.configure accepts EIP-712 signatures from the current permissionSigner and only requires the account be kernel.registered; it does not require that the permission be registered. Template config nonces are local to the template and are not rotated or cleared by SailKernel revoke/replace/session operations. As a result, old-but-unexpired configure signatures can be executed while the permission is inactive. When the same permission address is later re-registered (with a fresh kernel signerSig), a malicious manager can ensure the stale, broader configuration is installed at re-enable time and immediately dispatch/batch under those broader rules to move funds.

In ConfigurablePermission, configure() verifies (i) the account is kernel.registered(account) and (ii) the signature is by the current permissionSigner (read via kernel.configs(account)) against the template-local nonce configNonces[account]. It does not check whether the permission is currently registered for that account. Template configuration is stored per-account in the template's own storage and the template's config nonce is incremented only when configure or setAgentIdentity succeeds. In SailKernel, registerPermission/revokePermission/replacePermission are authorized via separate signerNonces per account; these kernel operations also bump managerNonces and batchNonces epochs on restriction events, but they never touch template-local config nonces or state. Dispatch/dispatchBatch require the named permission address be registered, but they rely on the permission's evaluate/evaluateBatch reading the template's stored configuration – whatever was last written. Consequently, a previously signed configure (still within its deadline and matching the current template nonce) can be executed even while the permission is unregistered. If the operator later re-registers the same permission address (fresh registerPermission signerSig), a malicious manager who controls submission order can apply the stale/broader config immediately before (or alongside) re-registration and

then dispatch/batch under those broadened rules, moving funds contrary to the operator's intent.

file: contracts/templates/ConfigurablePermission.sol

```
22  /// @notice UNAUDITED EXAMPLE – NOT PART OF THE TRUSTED CORE.

23  ///          Base class for the unaudited reference example
permissions (SwapPermission,
24  ///          BorrowPermission, TransferPermission, ApproveAndCall-
BatchPermission,
...

41  bytes32 public constant CONFIGURE_TYPEHASH =
42  - keccak256("Configure(address account,bytes32
paramsHash,uint256 nonce,uint256 deadline)");

43
44  bytes32 public constant SET_IDENTITY_TYPEHASH =
...

84  if (!kernel.registered(account)) revert AccountNotRegis-
tered(account);
85

86  // Read nonce before incrementing – nonce is incremented
AFTER signature verification
87  // succeeds, preventing a failed verify from consuming
the nonce.
```



```

...

102 // Increment nonce only after successful verification
103 configNonces[account] = nonce + 1;

104 _applyConfig(account, params);
105 isConfigured[account] = true;

...

113 if (msg.sender != permSigner) revert NotPermissionSign-
114 er(msg.sender, permSigner);

115 uint256 nonce = configNonces[account]++;
116 _applyConfig(account, params);

...

145 if (!kernel.registered(account)) revert AccountNotRegis-
146 tered(account);

147 uint256 nonce = configNonces[account];
148 bytes32 identityHash = keccak256(abi.encode(ref));

```

```

22 /// @notice UNAUDITED EXAMPLE – NOT PART OF THE TRUSTED CORE.
23 +/// SECURITY FIX GUIDANCE:
24 +/// - Bind configuration signatures to the kernel's signer-epoch
   so stale configs cannot be replayed
25 +/// across revoke/replace/session. Recommended pattern:
26 +/// 1) Extend ISailKernelView to expose `managerNonces(ad-
   dress)` and derive `epoch = managerNonces >> 128`.
27 +/// 2) Add `uint256 epoch` into the EIP-712 structs for
   Configure and SetAgentIdentity, and verify
28 +/// it equals the current epoch at submission time.
29 +/// 3) Track `configuredEpoch[account] = epoch` on successful
   configure()/configureDirect().
30 +/// 4) Add a base helper `_isConfigCurrent(account)` that
   compares `configuredEpoch` to the current
31 +

```



```

    ///         epoch and make every template's evaluate()/evaluate-
    Batch() return false when not current.
32 +///         5) Bump the EIP-712 domain version string to invalidate
    any previously issued configure signatures.
33 +/// - Optionally (stricter ops): require the permission to be
    registered at configure() time to disallow
34 +/// pre-configuration, but this is not necessary if epoch-binding
    is implemented correctly.
35 ///         Base class for the unaudited reference example
    permissions (SwapPermission,
36 ///         BorrowPermission, TransferPermission, ApproveAndCall-
    BatchPermission,
...

53     bytes32 public constant CONFIGURE_TYPEHASH =
54 +/// Include `epoch` (derived from kernel.managerNonces(ac-
    count) >> 128) in the typed data
55 +/// to bind config to the current kernel signer-epoch and
    prevent stale-config replay.
56 + keccak256("Configure(address account,bytes32
    paramsHash,uint256 nonce,uint256 deadline,uint256 epoch)");
57
58     bytes32 public constant SET_IDENTITY_TYPEHASH =
...

98     if (!kernel.registered(account)) revert AccountNotRegis-
    tered(account);
99
100 +/// SECURITY: derive current epoch = kernel.managerNonces(ac-
    count) >> 128 and include it in
101 +/// the EIP-712 struct. Reject signatures whose epoch !=
    current to prevent stale replay.
102 +/// Also record `configuredEpoch[account] = epoch` after
    successful verification.
103 // Read nonce before incrementing – nonce is incremented
    AFTER signature verification
104 // succeeds, preventing a failed verify from consuming
    the nonce.
...

119 // Increment nonce only after successful verification
120     configNonces[account] = nonce + 1;
121 +/// configuredEpoch[account] = epoch; // store the epoch
    used for this config

```

```

122 _applyConfig(account, params);
123   isConfigured[account] = true;

...

131   if (msg.sender != permSigner) revert NotPermissionSign-
    er(msg.sender, permSigner);
132
133 +// SECURITY: set `configuredEpoch[account] = (kernel.man-
    agerNonces(account) >> 128)` here.
134 +// This keeps direct configs current with the kernel's
    signer-epoch.
135   uint256 nonce = configNonces[account]++;
136   _applyConfig(account, params);

...

165   if (!kernel.registered(account)) revert AccountNotRegis-
    tered(account);
166
167 +// SECURITY: include current epoch in the typed data for
    identity as well and set
168 +// `configuredEpoch[account] = epoch` on success to keep
    state aligned with the kernel epoch.
169 +// This prevents stale identity writes across epoch
    rotations too.
170   uint256 nonce = configNonces[account];
171   bytes32 identityHash = keccak256(abi.encode(ref));

```

Severity

Impact Explanation: [High] Enables direct, unauthorized movement of principal funds from the victim Safe by re-enabling a permission under stale/broader rules and immediately dispatching/batch-dispatching to drain assets.

Likelihood Explanation: [Medium] Exploitation requires the presence of an old-but-unexpired configure signature that matches the current template nonce and a re-registration of the same permission address, plus delegated submission control. These are significant but realistic constraints in common operational patterns; no admin malice or integration failure is needed.

Exploitation

Exploitation Scenarios:

Scenario 1.

ApproveAndCallBatchPermission stale-config replay to bypass recipient pin: An old, unused configure signature sets `requireRecipientIsAccount=false` (default) with allowed token, spender, and consuming (router, selector) pair. The permission had been revoked. The `permissionSigner` later provides a fresh `registerPermission` signature to re-enable the same address with the intention to set `requireRecipientIsAccount=true`. The adversarial manager first submits the stale configure to install recipient-unconstrained mode, then registers the permission, and immediately calls `dispatchBatch` with `[approve, swap to attacker address, approve(0)]`. The batch evaluates true and assets are delivered to the attacker.

Preconditions / Assumptions

- (a). Account is `kernel.registered` and uses `ApproveAndCallBatchPermission`
- (b). A prior unused configure signature exists for this template/account with `requireRecipientIsAccount=false`, valid deadline, and matching current template config nonce
- (c). Permission was revoked and will be re-registered at the same permission address (fresh `registerPermission` signer-Sig)



(d). Adversarial manager holds the stale configure signature and the fresh registerPermission signature and controls submission order

Scenario 2.

TransferPermission stale recipient drain: An old, unused configure signature allowlists a valuable token and an attacker-controlled recipient (e.g., historical manager wallet) with a large cap. The permission is currently revoked. The permissionSigner provides a fresh registerPermission signature to re-enable the same address expecting a tighter config. The adversarial manager first submits the stale configure, then registers the permission, then dispatches ERC-20.transfer to the attacker recipient within the stale cap. The template approves and funds are moved out.

Preconditions / Assumptions

- (a). Account is kernel.registered and uses TransferPermission
- (b). A prior unused configure signature exists with isAllowedToken including a valuable token, isAllowedRecipient including an attacker-controlled address, and a sufficiently large cap; the signature is unexpired and matches current template config nonce
- (c). Permission was revoked and will be re-registered at the same permission address (fresh registerPermission signerSig)
- (d). Adversarial manager holds the stale configure signature and the fresh registerPermission signature and controls submission order

Scenario 3.

WithdrawPermission stale pinned recipient drain: An old, unused configure signature allowlists valuable tokens and sets allowedRecipient to an attacker-controlled address with a large cap. After stale configure is submitted and the permission is re-registered using a fresh signerSig, the adversarial manager dispatches ERC-20.transfer or transferFrom (from=account) to the stale allowedRecipient. The template authorizes under stale config and tokens are transferred to the attacker.

Preconditions / Assumptions

- (a). Account is kernel.registered and uses WithdrawPermission
- (b). A prior unused configure signature exists with isAllowedToken including a valuable token and allowedRecipient set to an attacker-controlled address with a suffi-



ciently large cap; the signature is unexpired and matches current template config nonce

(c). Permission was revoked and will be re-registered at the same permission address (fresh registerPermission signer-Sig)

(d). Adversarial manager holds the stale configure signature and the fresh registerPermission signature and controls submission order

Octane Security

Vulnerability #9

Missing trusted singleton check in `SailKernel.registerAccount` causes fee theft and permission bypass

Severity: HIGH

Likelihood: MEDIUM

Impact: HIGH

Status: Resolved

Fixed in PR #53. `registerAccount` verifies the Safe's `masterCopy()` against the governance trusted-singleton allowlist, ordered ahead of any other `ISafe` call so no method is invoked on an unverified singleton.

`SailKernel.registerAccount` verifies only the `SafeProxy` runtime codehash and a boolean `isModuleEnabled`, but never binds the proxy to a trusted Safe singleton. A malicious manager can register a genuine `SafeProxy` that delegates to a malicious singleton which forges Safe module execution and return data. The kernel then trusts falsified results in `collectFees/dispatch`, enabling protocol fee theft and bypass of permission gating.

The self-registration flow (`registerAccount`) accepts any caller whose runtime codehash matches the allowlisted Gnosis `SafeProxy v1.4.1` and for which `ISafe.isModuleEnabled(address(this))` returns true. It does not verify the proxy's singleton (master copy) against `governance.trustedSafeSingleton`, nor does it pin the singleton's codehash. Because `SafeProxy` delegates all logic to its singleton, a malicious manager can deploy a `SafeProxy` pointing to a hostile singleton that: always reports the kernel as enabled, fabricates success/return data for `execTransactionFromModule` and `execTransactionFromModuleReturnData`, ignores the provided target/data and executes attacker-chosen calls.

The kernel relies on Safe's module semantics for both fee transfers and call execution. In `collectFees`, it trusts the module's (ok, `returnData`) to confirm ETH/ERC-20 transfers to treasury and distributor; a malicious singleton can fake these while paying only the manager's share. In `dispatch/dispatchBatch`, the kernel evaluates permissions over a specific target/data sequence, but the malicious singleton can ignore those parameters and perform arbitrary attacker-chosen calls while returning success. This breaks the core invariant that registered accounts are genuine Safes with trusted module behavior and that the kernel constrains a malicious manager.

file: `contracts/core/SailKernel.sol`



```

651  /// @param feePolicy      Fee policy contract; address(0)
    = no fee policy.
652  /// @param feeAsset      Canonical fee settlement token;
    address(0) = native ETH.

653  function registerAccount(address permissionSigner, address
manager, address feePolicy, address feeAsset) external {
654  bytes32 codehash;

...

1138 /// @param managerSig EIP-712 signature over Dispatch struct
    by the account's manager.
1139 /// @param deadline    Unix timestamp after which the signature
    is invalid.

1140 function dispatch(
1141 address account,

...

1255 /// @param managerSig EIP-712 signature over DispatchBatch
    struct by the account's manager.
1256 /// @param deadline    Unix timestamp after which the signature
    is invalid.

1257 function dispatchBatch(
1258 address account,

...

1484 /// @dev DENOMINATION WARNING: `grossFee` and `currentNav`
    must be expressed in the same
1485 ///      token units as the fee token (or ETH wei if
    feeToken==address(0)). Mixing

1486 ///      denominations between grossFee and currentNav will
    silently produce incorrect fee
1487 ///      calculations inside the policy.

```



```

651  /// @param feePolicy      Fee policy contract; address(0)
    = no fee policy.
652  /// @param feeAsset      Canonical fee settlement token;
    address(0) = native ETH.
653 +// SECURITY FIX NEEDED:
654 +// Enforce that the caller SafeProxy delegates to a trusted
    singleton. Read masterCopy() via the proxy
655 +// fallback (selector 0xa619486e) and require governance.trust-
    edSafeSingleton(singleton) before registering.
656  function registerAccount(address permissionSigner, address
    manager, address feePolicy, address feeAsset) external {
657  bytes32 codehash;

...

1141 /// @param managerSig  EIP-712 signature over Dispatch struct
    by the account's manager.
1142 /// @param deadline    Unix timestamp after which the signature
    is invalid.
1143+// SECURITY FIX NEEDED: Before proceeding, assert `account`
    is a SafeProxy and its masterCopy() is a
1144+// governance-trusted Safe singleton (per-call guard to protect
    legacy registrations).
1145  function dispatch(
1146  address account,

...

1260 /// @param managerSig  EIP-712 signature over DispatchBatch
    struct by the account's manager.
1261 /// @param deadline    Unix timestamp after which the signature
    is invalid.
1262+// SECURITY FIX NEEDED: Before proceeding, assert `account`
    is a SafeProxy and its masterCopy() is a
1263+// governance-trusted Safe singleton (per-call guard to protect
    legacy registrations).
1264  function dispatchBatch(
1265  address account,

...

1491 /// @dev DENOMINATION WARNING: `grossFee` and `currentNav`
    must be expressed in the same
1492 ///      token units as the fee token (or ETH wei if
    feeToken==address(0)). Mixing
1493+// SECURITY FIX NEEDED: Before proceeding, assert `account`
    is a SafeProxy and its masterCopy() is a

```

```
1494+// governance-trusted Safe singleton (per-call guard to protect
    legacy registrations).
1495 ///      denominations between grossFee and currentNav will
    silently produce incorrect fee
1496 ///      calculations inside the policy.
```

Octane Security

Severity

Impact Explanation: [High] Scenarios 2 and 3 enable direct theft of principal funds and break core invariants about permission-constrained execution; Scenario 1 causes direct, material loss of protocol/distributor fees. The maximum impact across scenarios is high.

Likelihood Explanation: [Medium] Registering a SafeProxy with a malicious singleton via self-registration has no special technical constraints. Scenario 1 is commonly feasible in self-managed deployments (manager-controlled fee policy), approaching high likelihood. Scenarios 2 and 3 additionally require depositors to fund the attacker's registered account, a constraint outside attacker's exclusive control but plausible under intended usage; thus overall likelihood is medium.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (protocol/distributor fee theft): The attacker deploys a malicious singleton `M` that (a) always reports the kernel module as enabled and (b) forges success/return data in `execTransactionFromModule{ReturnData}`. They deploy a SafeProxy `P` pointing to `M` and call `SailKernel.registerAccount` from `P`; checks pass (trusted SafeProxy runtime codehash and `isModuleEnabled == true`). Using an allowlisted fee policy they control, they seed `HWM` and call `collectFees`. `M` fakes success for the treasury/distributor transfers (no funds move) while performing only the `managerTake` transfer. The kernel emits `FeesCollected` and advances policy state; protocol/distributor revenue is silently stolen.

Preconditions / Assumptions

- (a). Governance has allowlisted the canonical SafeProxy v1.4.1 runtime codehash in `trustedSafeProxyCodehash`
- (b). `SailKernel.registerAccount` is available and used (self-registration path)
- (c). Malicious singleton `M` implements Safe module interface functions and can forge `isModuleEnabled` and `exec-`



TransactionFromModule{ReturnData} semantics

(d). An allowlisted fee policy is set for the account; in self-managed deployments, attacker controls feeManager and seeds HWM

(e). Account holds sufficient feeAsset balance to call collectFees

Scenario 2.

Scenario 2 (permission bypass and principal theft): The attacker registers P (malicious singleton) as above, sets their own permissionSigner/manager, and registers a permission. They submit a manager-signed dispatch that the permission would approve for a benign target/data. The kernel verifies the signature and permission, then calls ISafe.execTransactionFromModule. M ignores the vetted target/data and executes attacker-chosen calls (e.g., transfer all assets to the attacker), returning success. The kernel logs success while funds are drained.

Preconditions / Assumptions

(a). Governance has allowlisted the canonical SafeProxy v1.4.1 runtime codehash in trustedSafeProxyCodehash

(b). SailKernel.registerAccount is available and used (self-registration path)

(c). Malicious singleton M can overwrite/ignore to/value/data and return forged success

(d). Attacker controls permissionSigner/manager for the registered account and registers a permission

(e). Depositors send assets to the registered account (intended protocol usage)

Scenario 3.

Scenario 3 (batch tampering/cleanup violations): The attacker registers P (malicious singleton) and uses dispatchBatch with a sequence that a batch-aware permission would approve (e.g., approve, swap, reset). During execution, M rewrites/omits subcalls (e.g., redirects swap to attacker pool, skips reset approvals) and returns success for all subcalls. The kernel emits BatchDispatched; invariants and cleanup that permissions intend are nullified, enabling stealthy asset diversion or dangerous lingering approvals.

Preconditions / Assumptions

(a). Governance has allowlisted the canonical SafeProxy v1.4.1 runtime codehash in trustedSafeProxyCodehash

(b). SailKernel.registerAccount is available and used (self-registration path)

(c). Malicious singleton M can reorder/omit/alter subcalls in execTransactionFromModule while returning success



- (d). Attacker controls permissionSigner/manager and registers a batch-aware permission on the account
- (e). Depositors send assets to the registered account (intended protocol usage)

Octane Security

Vulnerability #10

Unnetted NAV–HWM performance fee in StandardFeePolicy causes direct principal loss from deposits/airdrops

Severity: HIGH

Likelihood: MEDIUM

Impact: HIGH

Status: Acknowledged

Accepted by design. The reference StandardFeePolicy uses a manager-attested NAV model, and the kernel deliberately neither computes nor validates NAV – this keeps the trusted core minimal and venue-agnostic; NAV measurement is the manager's responsibility (or that of future audited, NAV-validating policies). A performance fee can therefore be crystallised on any reported NAV rise, including fresh deposits/airdrops. The no-external-loss property holds when `feeManager == owner` for self-managed SMAs (the launch scope); third-party allocation requires a NAV-validating IFeePolicy, which arrives with the Marketplace phase. The acceptance rationale was corrected in PR #67 (the prior "`manager == owner @owner→owner`" framing was inaccurate – the manager take is paid to the policy's `feeManager`, not the owner).

StandardFeePolicy charges performance fees on any NAV increase over the high-water mark without netting deposits or other inflows, allowing the manager to extract fees from fresh principal or windfalls. SailKernel enforces only the policy's `maxFee` ceiling and then transfers funds, resulting in direct, material loss to the account despite honest NAV reporting.

StandardFeePolicy.computeFee calculates fees per call as: (1) `managementFee = currentNav × appliedManagementFeeBps × elapsed / (365 days × 10,000)` and (2) `performanceFee = max(currentNav - highWaterMark[account], 0) × appliedPerformanceFeeBps / 10,000`. The high-water mark (HWM) is updated in `recordCollection` to `max(HWM, currentNav)` after enforcing a 1-day minimum interval. The policy does not net out deposits, airdrops, or other non-performance inflows that occurred since the last collection. As a result, any NAV increase above HWM, even if caused solely by fresh principal or windfalls, is treated as “profit” and charged a performance fee. `SailKernel.collectFees` accepts manager-submitted `currentNav`, calls `computeFee` for a ceiling, then enforces `grossFee ≤ maxFee`, calls `recordCollection`, and pays out protocol/distributor/manager splits. With honest and consistent-unit NAV, a manager can time collection immediately after an inflow to crystallize a performance fee on that inflow. While the management fee's snapshot-on-spot-NAV behavior is intentional, the core harmful effect here is the performance fee being charged on flows rather than true profit, leading to direct principal loss.

```

265     uint256 performanceFee;

...

266     uint256 hwm = highWaterMark[account];
267     if (currentNav > hwm) {
...
287 }
288

289     if (block.timestamp < lastCollectionTimestamp[account] +
MIN_COLLECTION_INTERVAL)
290         revert CollectionTooFrequent();
...

309     hwmSeeded[account] = true;
310     highWaterMark[account] = initialNav;

311     lastCollectionTimestamp[account] = block.timestamp;
312     appliedManagementFeeBps[account] = managementFeeBps;

```

```

265     uint256 performanceFee;
266 +// SECURITY: To prevent charging performance fees on net
inflows (deposits/airdrops),
267 +// implement a flow-adjusted HWM: baseline = HWM +
max(netInflows, 0), where
268 +// netInflows = (kernel.cumulativeDeposits(account) -
lastCumDeposits[account])
269 +// - (kernel.cumulativeWithdrawals(account) -
lastCumWithdrawals[account]).
270 +// Until implemented, performance fees may be applied to
fresh principal.
271     uint256 hwm = highWaterMark[account];
272     if (currentNav > hwm) {
...

```

```
292 }
293
294 +// FLOW-ADJUSTED HWM: snapshot kernel cumulativeDe-
    posits/Withdrawals here for next period.
295 +// e.g., lastCumDeposits[account] = kernel.cumulativeDe-
    posits(account); similarly for withdrawals.
296     if (block.timestamp < lastCollectionTimestamp[account] +
        MIN_COLLECTION_INTERVAL)
297         revert CollectionTooFrequent();

...

316     hwmSeeded[account]      = true;
317     highWaterMark[account]  = initialNav;
318 +// FLOW-ADJUSTED HWM: initialize cumulativeDeposits/With-
    draws snapshots here.
319     lastCollectionTimestamp[account] = block.timestamp;
320     appliedManagementFeeBps[account] = managementFeeBps;
```


Exploitation

Exploitation Scenarios:

Scenario 1.

After the last collection (HWM=1,000) and ≥ 1 day elapsed, the owner deposits 1,000 units into the Safe (no trading profit). Manager calls `collectFees` with `currentNav=2,000`. `computeFee` returns `performanceFee = (2,000-1,000) \times perfBps / 10,000`, i.e., a fee on the deposit. Kernel enforces the ceiling and transfers the fee out of the Safe, causing immediate principal loss.

Preconditions / Assumptions

- (a). Account uses `StandardFeePolicy` and it is allowlisted
- (b). `highWaterMark` seeded with non-zero initial NAV by `feeManager`
- (c). At least 1 day since last collection or seeding
- (d). `appliedPerformanceFeeBps > 0`
- (e). Manager is authorized to call `collectFees`
- (f). NAV is honest and denominated consistently with `feeAsset`
- (g). Safe holds enough `feeAsset/ETH` to pay
- (h). A post-collection owner deposit occurred

Scenario 2.

After the last collection and ≥ 1 day elapsed, the Safe receives an airdrop/donation that raises `currentNav` above HWM without investment gains. Manager calls `collectFees` with honest `currentNav`. `computeFee` treats the windfall as profit and returns a performance fee on that amount. Kernel transfers the fee out of the Safe, extracting principal.

Preconditions / Assumptions

- (a). Account uses `StandardFeePolicy` and it is allowlisted
- (b). `highWaterMark` seeded with non-zero initial NAV by `feeManager`
- (c). At least 1 day since last collection or seeding
- (d). `appliedPerformanceFeeBps > 0`
- (e). Manager is authorized to call `collectFees`
- (f). NAV is honest and denominated consistently with `feeAsset`
- (g). Safe holds enough `feeAsset/ETH` to pay
- (h). A post-collection exogenous inflow (airdrop/donation) occurred

Scenario 3.



Shortly before a collection (≥ 1 day since last collection; $HWM=1,000$), the owner deposits 1,000 units (no profit). Manager calls `collectFees` with `currentNav=2,000`. `computeFee` returns both `performanceFee` on the deposit and `managementFee` on spot NAV. Kernel transfers the combined fee, causing direct principal loss (performance fee dominates) plus the intended management fee component.

Preconditions / Assumptions

- (a). Account uses `StandardFeePolicy` and it is allowlisted
- (b). `highWaterMark` seeded with non-zero initial NAV by `feeManager`
- (c). At least 1 day since last collection or seeding
- (d). `appliedPerformanceFeeBps` > 0 and `appliedManagementFeeBps` > 0
- (e). Manager is authorized to call `collectFees`
- (f). NAV is honest and denominated consistently with `feeAsset`
- (g). Safe holds enough `feeAsset/ETH` to pay
- (h). A post-collection owner deposit occurred shortly before collection

Severity

Impact Explanation: [High] Kernel moves `feeAsset/ETH` out of the Safe based on `computeFee`'s ceiling, enabling direct, material loss of principal when performance fees are charged on deposits/airdrops treated as profit.

Likelihood Explanation: [Medium] Exploitation requires a normal but external precondition—an inflow (deposit or plausible airdrop) between collections and sufficient `feeAsset` at collection time. The manager can time collection to coincide with these common events.



Vulnerability #11

cToken-as-asset in BorrowPermission Compound path causes DoS of Compound borrows

Severity: MEDIUM

Likelihood: MEDIUM

Impact: MEDIUM

Status: Resolved

Fixed in PR #54. The operator allowlists the underlying asset (not the cToken); the template resolves and validates `ICERC20(target).underlying()`; cETH is fail-closed denied.

In BorrowPermission's Compound V2 branch, the template treats the cToken (`ctx.target`) as the borrow asset for both the asset allowlist and the borrow oracle, while the borrow amount is denominated in the underlying. If the borrow-oracle adapter expects an underlying token as base, prices resolve to 0/unsupported, causing all Compound borrows to be denied. If the adapter prices cTokens per share instead of normalizing to per-underlying-unit, LTV is miscomputed (typically understated), allowing over-LTV borrows to pass.

BorrowPermission's Compound V2 path decodes `borrow(uint256)` on the cToken and uses `ctx.target` (the cToken address) as the borrow "asset" for both the `assets[]` allowlist and the LTV check. It then calls `_ltvCheck` with `asset=ctx.target` and amount in underlying units. Inside `_ltvCheck`, the borrow oracle is queried as `getPrice(asset, address(0))` with `asset` set to the cToken, and the computed `borrowScaled = amount * borPrice / 10^borDec` assumes `borPrice` is a per-underlying-unit price. Under common oracle-adapter designs that expect the underlying token as base, calling with a cToken base returns 0/unsupported, causing the LTV check to fail and denying all Compound borrows (DoS). Alternatively, if the adapter returns a cToken-share price for the cToken base (not normalized to per-underlying-unit), the borrow value is understated (or overstated, depending on scaling), leading to under- (or over-) enforcement of the configured `maxLtvBps`. This behavior stems from the template's choice to use the cToken as the asset identifier in the Compound path while the borrow amount is specified in underlying units.

file: contracts/templates/BorrowPermission.sol

```
8  import {ConfigurablePermission} from "./ConfigurablePermission.sol";
9  import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
```

```

10
11  /// @title BorrowPermission – bounded borrow with optional LTV
    ceiling
...

181  if (txData.length < LEN_COMPOUND) return false;
182  uint256 amount = abi.decode(txData[4:], (uint256));
183 - if (!isAllowedAsset[ctx.account][ctx.target]) return
    false;

184  if (amount > s.maxAmountPerTx)          return
    false;
185 - return _ltvCheck(s, ctx.target, amount, ctx.account);
186  }

```

```

8  import {ConfigurablePermission} from "../ConfigurablePermis-
    sion.sol";
9  import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
10 +interface ICerc20 {
11 + function underlying() external view returns (address);
12 +}
13
14  /// @title BorrowPermission – bounded borrow with optional LTV
    ceiling
...

184  if (txData.length < LEN_COMPOUND) return false;
185  uint256 amount = abi.decode(txData[4:], (uint256));
186 + address underlying; try ICerc20(ctx.target).underlying()
    returns (address u) { underlying = u; } catch { return false; }
187 + if (!isAllowedAsset[ctx.account][underlying]) return
    false;
188  if (amount > s.maxAmountPerTx)          return
    false;
189 + return _ltvCheck(s, underlying, amount, ctx.account);
190  }

```

Severity

Impact Explanation: [Medium] Denies intended borrowing functionality on the Compound path (significant availability loss) and can mis-enforce LTV ceilings, but does not directly steal funds or break core protocol invariants.

Likelihood Explanation: [Medium] The DoS scenario requires a plausible, non-malicious adapter design (underlying-only base) and ordinary configuration; no special timing or complex setup is needed, though adapter semantics are outside attacker control.

Exploitation

Exploitation Scenarios:

Scenario 1.

DoS: Account configures BorrowPermission with oracles enabled and allowlists a Compound cToken market. The borrow-oracle adapter expects the underlying token as base and returns 0/unsupported for a cToken base. When the manager calls cToken.borrow(amount), evaluate() passes the cToken address into _ltvCheck; borrowOracle.getPrice(cToken, 0) returns 0, causing the LTV check to fail. Kernel reverts with PermissionDenied, making all Compound borrows through this permission unusable.

Preconditions / Assumptions

- (a). BorrowPermission is registered on the account and configured with both collateralOracle and borrowOracle and a non-zero maxPriceAgeSec
- (b). The Compound cToken market is allowlisted in both protocols] and assets[
- (c). The borrow-oracle adapter expects base=underlying and returns 0/unsupported for base=cToken
- (d). Manager submits a Compound borrow via cToken.borrow(amount)

Scenario 2.

Over-LTV approval: Same configuration, but the borrow-oracle adapter returns a cToken-share price for base=cToken (not normalized to per-underlying-unit). The amount is



in underlying units, so `borrowScaled` is understated. The computed LTV falls below `maxLtvBps` even when the true LTV exceeds it, allowing the manager to borrow above the configured ceiling and increasing liquidation risk.

Preconditions / Assumptions

- (a). `BorrowPermission` is registered and configured with both oracles and a non-zero `maxPriceAgeSec`
- (b). The Compound `cToken` market is allowlisted in both `protocols` and `assets`. The borrow-oracle adapter, when called with `base=cToken`, returns a `cToken`-share price (not normalized to per-underlying-unit)
- (d). Manager submits a Compound borrow via `cToken.borrow(amount)`

Scenario 3.

Overly strict denials: Same configuration, but the adapter's `cToken` pricing or scaling overstates borrow value. The computed LTV exceeds `maxLtvBps` for otherwise safe borrows, causing `evaluate()` to deny valid Compound borrows.

Preconditions / Assumptions

- (a). `BorrowPermission` is registered and configured with both oracles and a non-zero `maxPriceAgeSec`
- (b). The Compound `cToken` market is allowlisted in both `protocols` and `assets`. The borrow-oracle adapter's `cToken` pricing/scaling overstates borrow value relative to per-underlying-unit pricing
- (d). Manager submits a Compound borrow via `cToken.borrow(amount)`

Vulnerability #12

Missing cross-oracle contemporaneity in BorrowPermission LTV check causes policy under-enforcement and elevated liquidation risk

Severity: LOW

Likelihood: MEDIUM

Impact: LOW

Status: Acknowledged

Valid; deferred to the post-audit backlog. The robust fix changes the per-account config ABI and ripples into the Sailor toolkit, so it is scoped as a follow-up rather than a pre-freeze change. No funds at risk at launch under the self-managed model.

BorrowPermission forms an LTV ratio from two independent oracle reads (collateral value and borrow-asset price) and only enforces each feed's staleness individually. It does not require the two observations to be contemporaneous or within a bounded skew. A manager can time borrows during volatile moves so collateral is valued too high and/or the borrow asset too low (both still within `maxPriceAgeSec`), causing the computed LTV to understate true leverage and allowing a borrow above the intended ceiling.

In `BorrowPermission._ltvCheck`, the contract fetches the aggregate collateral valuation via `IOracle.getPrice(account, address(0))` and the borrow-asset price via `IOracle.getPrice(asset, address(0))`. It validates freshness for each feed independently as $\text{block.timestamp} - \text{updatedAt} \leq \text{maxPriceAgeSec}$ and then computes LTV from these two values. There is no requirement that the two observations are contemporaneous or within a bounded skew; only per-feed staleness is enforced. As a result, during rapid market moves or when adapters update at different cadences, a manager can submit a borrow while the collateral oracle is still fresh-high and/or the borrow oracle is fresh-low, both within the freshness window. The computed LTV can fall below `maxLtvBps` even though the true contemporaneous ratio would exceed it, under-enforcing the operator's per-call LTV ceiling. The kernel then executes the borrow since `evaluate()` returns true. Underlying lending protocols (Aave/Morpho/Compound) still enforce their own health-factor/LTV via their oracles, so this does not directly bypass venue limits. Impact is elevated liquidation risk and defeat of the operator's intended per-call safety margin. This behavior is consistent with documented semantics (per-feed staleness only) but remains exploitable by timing.

file: contracts/templates/BorrowPermission.sol

```
68     uint256 private constant LEN_COMPOUND = 36;
```

69

```
70     struct Slot {
71         address[] protocols;

...

124     uint256 maxPriceAgeSec
125 ) = abi.decode(params, (address[], address[], uint256,
    uint256, address, address, uint256));

126
127     if (maxLtvBps > 10_000) revert LtvBpsTooLarge(maxLtvBps);

...

209     if (borUpdatedAt == 0 || block.timestamp - borUpdatedAt
210         > s.maxPriceAgeSec) return false;
    }

211
212     if (colDec > 77 || borDec > 77) return false;
```

```
68     uint256 private constant LEN_COMPOUND = 36;
69
70     +// FIX GUIDE: To mitigate cross-oracle skew, extend Slot with
    per-account fields such as:
71     +//     uint256 maxSkewSec; uint256 colHaircutBps; uint256
    borMarkupBps;
72     +// and plumb them through getConfig/_applyConfig. See _ltvCheck
    for the enforcement point.
73     struct Slot {
74         address[] protocols;

...

127     uint256 maxPriceAgeSec
128 ) = abi.decode(params, (address[], address[], uint256,
    uint256, address, address, uint256));
```



```

129 +// FIX GUIDE: Extend abi.decode to include new fields,
    validate bps d10_000, and store them.
130 +// Optionally require a non-zero maxSkewSec when oracles
    are set.
131
132   if (maxLtvBps > 10_000) revert LtvBpsTooLarge(maxLtvBps);
...

214   if (borUpdatedAt == 0 || block.timestamp - borUpdatedAt
    > s.maxPriceAgeSec) return false;
215 }
216 +// FIX GUIDE: Enforce cross-oracle contemporaneity:
217 +// if (s.maxSkewSec > 0) { uint256 dt = colUpdatedAt
    > borUpdatedAt ? (colUpdatedAt - borUpdatedAt) : (borUpdatedAt -
    colUpdatedAt); if (dt > s.maxSkewSec) return false; }
218 +// Optionally apply conservative margins before LTV:
219 +// colValue = Math.mulDiv(colValue, 10_000 - s.col-
    HaircutBps, 10_000); borPrice = Math.mulDiv(borPrice, 10_000 +
    s.borMarkupBps, 10_000);
220
221   if (colDec > 77 || borDec > 77) return false;

```

Severity

Impact Explanation: [Low] The issue under-enforces a per-call LTV ceiling during volatile moves, increasing liquidation risk for the account, but does not directly cause principal loss, theft, or freeze. Underlying lending protocols still enforce their own HF/LTV.

Likelihood Explanation: [Medium] Exploitation requires timing vs independent oracle update cadences during volatility and monitoring of updatedAt; these are significant but realistic constraints outside the attacker's control. Dual-skew overlap is rarer (lower likelihood), but single-skew timing is plausible.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (borrow-asset spikes up; borrow oracle stale-low; collateral oracle fresh): The account has BorrowPermission with both oracles configured and maxPriceAgeSec > 0. Market moves the borrow asset from 2,000 to 3,000 in the shared numeraire. The borrow oracle is within-bound stale (2,000), while the collateral oracle is fresh. The manager monitors both adapters and submits a borrow sized to pass LTV using 2,000. BorrowPermission accepts (both feeds individually fresh), and the kernel executes the loan. True LTV at 3,000 would have exceeded the configured ceiling, reducing safety margin and increasing liquidation risk.

Preconditions / Assumptions

- (a). BorrowPermission registered for the account and selected in dispatch
- (b). Target lending protocol and borrow asset are allowlisted
- (c). Both collateralOracle and borrowOracle configured with shared numeraire
- (d). maxPriceAgeSec > 0 (required by configure when oracles are set)
- (e). Adapters are honest but may have differing ca-



- dences/lag; updatedAt reflects real update time
- (f). Sufficient market volatility to create meaningful divergence within the freshness window
- (g). Manager is adversarial and can monitor and time dispatches
- (h). Borrow amount chosen to also satisfy the lending venue's own HF/LTV

Scenario 2.

Scenario 2 (collateral drops; collateral oracle stale-high; borrow oracle fresh): The account is configured as above. The aggregate collateral value falls from 220,000 to 180,000, but the collateral oracle remains within-bound stale-high (220,000). The manager times a borrow so the computed LTV using 220,000 is just under maxLtvBps, while the true LTV at 180,000 would exceed it. BorrowPermission authorizes and the kernel executes, under-enforcing the intended ceiling.

Preconditions / Assumptions

- (a). BorrowPermission registered and selected in dispatch
- (b). Lending protocol and asset allowlisted
- (c). Both oracles configured with shared numeraire and maxPriceAgeSec > 0
- (d). Collateral oracle aggregates portfolio value by account and may update on a different cadence
- (e). Market decline in collateral value within freshness window; collateral oracle stale-high but within bound
- (f). Manager times the borrow during stale-high window and picks amount passing venue HF

Scenario 3.

Scenario 3 (double skew overlap: collateral stale-high and borrow stale-low): Within maxPriceAgeSec, collateralOracle reports 220,000 (stale-high) and borrowOracle reports 2,000 (stale-low) while true values are 180,000 and 3,000. The manager borrows near the computed cap; the check passes with stale values but would fail with contemporaneous ones. This is the most severe under-enforcement but requires overlapping skew windows.

Preconditions / Assumptions

- (a). Same as Scenarios 1-2
- (b). Overlapping skew windows: collateral oracle stale-high and borrow oracle stale-low simultaneously within maxPriceAgeSec
- (c). Manager monitors both feeds and times the borrow during the overlap



Octane Security



Vulnerability #13

Mixed-unit per-tx cap in DepositPermission ERC-4626 mint path causes assets-per-tx cap circumvention

Severity: LOW

Likelihood: MEDIUM

Impact: LOW

Status: Acknowledged

By design. The mint(shares) path bounds shares, not assets; deposit/assets) and both Aave paths cap assets correctly. An asset-denominated cap on the mint path would reintroduce a vault price-read into a deliberately oracle-free template. Shares remain bounded; no drain. Documented.

DepositPermission reuses a single maxAmountPerTx across deposit shapes with different units: ERC-4626 deposit checks assets against the cap while ERC-4626 mint checks shares. Because allowlisting an ERC-4626 vault authorizes both selectors and there is no per-selector cap or disable, a manager can use mint(shares) to spend more underlying than an assets-denominated cap would suggest when share price > 1 or decimals differ.

DepositPermission stores one per-account cap (maxAmountPerTx) and approves exactly four deposit shapes. For ERC-4626, deposit(uint256 assets,address) enforces assets 1 (convertToAssets(shares) > shares) or when share/underlying decimals differ, a manager can call mint(shares) with shares <= maxAmountPerTx but cause the vault to pull more underlying assets than the same cap would have allowed on the deposit/assets,...) path. The receiver is strictly the account, so there is no theft vector; the impact is exceeding an assets-per-transaction risk bound that operators may intend. ApproveAndCallBatchPermission does not mitigate this: it can ensure allowance and recipient checks but does not convert shares to assets; thus a batch can still execute a mint that exceeds an assets-based intent. This behavior is explicitly documented in the template (mint cap is in shares), making it a design limitation rather than an unintended bug, but it remains a practical avenue for a manager (treated as potentially adversarial within permissions) to exceed an assets-per-tx notion of the cap.

file: contracts/templates/DepositPermission.sol

```
6  import {SailCapabilities} from "../interfaces/SailCapabilities.sol";
7  import {ConfigurablePermission} from "../ConfigurablePermission.sol";
```

```
8
9  /// @title DepositPermission – bounded ERC-20 deposit credited
   to the account

...

148 // Asset is not in calldata; require the vault itself
    to be token-allowlisted.
149 if (!isAllowedToken[ctx.account][ctx.target]) return
    false;
150 -// NOTE: the cap on mint() is denominated in SHARES,
    not underlying assets – its
151 -// asset/USD value floats with the share price. These
    templates are intentionally
152 -// oracle-free; operators sizing this cap must account
    for the share price.
153 - if (shares > s.maxAmountPerTx) return false;
154 return receiver == ctx.account;
155 }
```

```

6  import {SailCapabilities} from "../interfaces/SailCapabili-
    ties.sol";
7  import {ConfigurablePermission} from "../ConfigurablePermis-
    sion.sol";
8  +interface IERC4626 { function previewMint(uint256) external view
    returns (uint256); }
9
10  /// @title DepositPermission – bounded ERC-20 deposit credited
    to the account
...

149  // Asset is not in calldata; require the vault itself
    to be token-allowlisted.
150  if (!isAllowedToken[ctx.account][ctx.target]) return
    false;
151  +// Enforce assets-denominated cap on mint via
    previewMint; revert 'denial by kernel.
152  + uint256 assets = IERC4626(ctx.target).pre-
    viewMint(shares);
153  + if (assets > s.maxAmountPerTx) return false;

154  return receiver == ctx.account;
155  }

```

Severity

Impact Explanation: [Low] No theft or DoS occurs; funds are credited as shares to the same account. The impact is a correctness/policy-bound gap where a single cap is applied inconsistently across selectors, enabling a manager to exceed an assets-per-transaction intent.

Likelihood Explanation: [Medium] ERC-4626 share price drift above 1 is common; managers can freely choose mint within registered permissions. Preconditions like sufficient balance/allowance and registration of the relevant permissions are typical for deposit workflows. There is no direct profit incentive, but no rare states are required.

Exploitation

Exploitation Scenarios:

Scenario 1.

Price-drift overshoot via ERC-4626 mint: With DepositPermission configured (targets=[Vault], tokens=[Vault], maxAmountPerTx=100e18) and share price at 1.5 assets/share, the manager dispatches Vault.mint(100e18, account). The permission checks shares 1 (e.g., 1.5 assets/share) (g). The account has sufficient underlying balance and allowance to the vault

Scenario 2.

Decimals-mismatch overshoot: With DepositPermission configured as above and a vault where shares and underlying have different decimals (e.g., shares 6, underlying 18), the manager calls Vault.mint(100e18, account). The permission compares shares to the cap, while convertToAssets implies far more underlying than a 100-asset cap would suggest, if balance and allowance suffice.

Preconditions / Assumptions

- (a). DepositPermission is registered and configured with the vault allowlisted in targets and tokens
- (b). ERC-4626 vault uses different decimals for shares vs underlying (permitted by the standard)
- (c). Session active; manager key valid



- (d). The account has sufficient underlying balance and allowance
- (e). Vault price around 1:1 (or any price; decimals mismatch amplifies asset usage)

Scenario 3.

Approval bracket then mint: With `ApproveAndCallBatchPermission` registered to allow (`target=vault`, `selector=mint`) and adequate per-token approval cap, the manager submits a batch [`approve(underlying, vault, A)`, `vault.mint(100e18, account)`, `approve(underlying, vault, 0)`]. The bracket ensures allowance and optionally `recipient==account`, but still permits a mint that exceeds an `assets-per-tx` intent when `share price > 1`.

Preconditions / Assumptions

- (a). `ApproveAndCallBatchPermission` is registered for the account
- (b). Its config allows the underlying token, `spender=vault`, and consuming pair (`vault`, `mint`)
- (c). Per-token `maxApprovalAmount` is sufficient for the intended mint
- (d). Optional: `requireRecipientIsAccount=true` (the template can decode ERC-4626 mint receiver)
- (e). Session active; manager key valid
- (f). ERC-4626 share price > 1
- (g). The account has sufficient underlying balance



Vulnerability #14

Replayable manager signatures due to nonces not burned on failure and unbound submitter in `SailKernel.dispatch/dispatchBatch` cause third-party timing control/MEV

Severity: LOW

Likelihood: MEDIUM

Impact: LOW

Status: Acknowledged

Accepted design tradeoff. The transaction submitter is intentionally unconstrained (whitepaper §2.1) – authority derives solely from the manager's EIP-712 signature – which is what makes the protocol compatible with relayers, paymasters, and ERC-4337 bundlers. Nonces consume only on success; replay stays within the signed envelope (recipients/params fixed, permission re-evaluated against live state), so the residual is execution timing/quality (MEV) with no permission bypass and no fund redirection – hence Low. Submitter-binding would break the stated design goal; burn-on-failure / per-nonce-cancel add audited surface for a Low and are redundant with existing coarse cancellation (`revokeSession` / `revoke` / `replace`). Operational mitigation (short signature deadlines + private-relay submission) lives in the Sailor toolkit. Note: PR #56 (`activateSession` epoch rotation) incidentally narrows this.

Manager-signed dispatches can be replayed by any observer until the deadline because manager/batch nonces are only consumed on successful execution and the signature does not bind the submitter. Permissions are evaluated at execution time against live state, so a seen signature functions as a resting order. This enables attacker-controlled timing within the signed bounds (e.g., MEV sandwiching swaps, timing borrows near LTV thresholds, and front-running in-place configuration tightens) without bypassing permission constraints.

`SailKernel`'s `dispatch` and `dispatchBatch` verify an EIP-712 manager signature over account, named permission, target/callsHash, value/data hash, current nonce, and deadline. The submitter (`msg.sender`) is not included in the signed struct. After signature verification, the kernel increments the manager/batch nonce, but any subsequent revert (e.g., permission denial, batch subcall failure, Safe module failure) rolls back the state change, so failed attempts do not consume the nonce. Because of this, any party that observes a valid signature (commonly via a reverted attempt) can keep replaying the exact same signed message until it succeeds or the deadline expires. Permissions are intentionally evaluated at execution time via `staticcall` with a gas cap and fail-closed semantics, and many templates read mutable state (e.g., oracle prices, allowlists, caps). As a result, an attacker who has seen a valid signature can choose when, within the deadline, to execute it under conditions that satisfy the permission's checks. This enables third-party timing control and MEV extraction (e.g., sandwiching



a swap) and allows a one-time execution just before an in-place configuration tighten is mined. The behavior does not allow bypass of permission constraints, nor does it redirect funds to unauthorized recipients; actions still follow the manager's signed calldata and permission rules. Coarse-grained revocations (replace/revoke permissions, revoke session) epoch-bump nonces to invalidate outstanding signatures, and short deadlines or private relays can mitigate exposure, but absent these, replay-until-success remains possible.

file: contracts/core/SailKernel.sol

```
98     uint256 private constant NONCE_EPOCH_INCREMENT = 1 << 128;
99
100    /// @dev ERC-1271 magic value returned by `isValidSignature`
    for a valid signature.
101    bytes4 private constant ERC1271_MAGIC           = 0x1626ba7e;
...
105    //
    -----
106
107    /// @notice EIP-712 type hash for manager dispatch authorisa-
    tion.
108    ///         Type string: "Dispatch(address account,address
    permission,address target,uint256 value,bytes32 dataHash,uint256
    nonce,uint256 deadline)"
...
1138   /// @param managerSig EIP-712 signature over Dispatch struct
    by the account's manager.
1139   /// @param deadline   Unix timestamp after which the signature
    is invalid.

1140   function dispatch(
1141   address account,
...

1255   /// @param managerSig EIP-712 signature over DispatchBatch
    struct by the account's manager.
```

```
1256 /// @param deadline    Unix timestamp after which the signature
    is invalid.
```

```
1257 function dispatchBatch(
1258     address account,
```

```
98     uint256 private constant NONCE_EPOCH_INCREMENT = 1 << 128;
99
100 +// SECURITY: Add manager-signed cancelDispatchNonce/cancel-
    BatchNonce ops to burn the current nonce without executing (allowed
    during pause).
101 /// @dev ERC-1271 magic value returned by `isValidSignature`
    for a valid signature.
102     bytes4 private constant ERC1271_MAGIC             = 0x1626ba7e;
...
106 //
    -----
107
108 +// SECURITY: Add v2 EIP-712 typehashes that include a
    'submitter' address and new dispatch2/dispatchBatch2
109 +// entrypoints requiring msg.sender == submitter to prevent
    third-party replay of observed signatures.
110 /// @notice EIP-712 type hash for manager dispatch authorisa-
    tion.
111 ///     Type string: "Dispatch(address account,address
    permission,address target,uint256 value,bytes32 dataHash,uint256
    nonce,uint256 deadline)"
...
1141 /// @param managerSig    EIP-712 signature over Dispatch struct
    by the account's manager.
1142 /// @param deadline    Unix timestamp after which the signature
    is invalid.
1143 +// SECURITY: Optionally add dispatchOneShot(...) variant that
    consumes the nonce even if Safe execution fails
1144 +// (no revert on failure; emit a failure event) to enforce
    single-use semantics for a signature.
1145     function dispatch(
1146         address account,
```

```
1260 /// @param managerSig EIP-712 signature over DispatchBatch
    struct by the account's manager.
1261 /// @param deadline    Unix timestamp after which the signature
    is invalid.
1262+// SECURITY: Optionally add dispatchBatchOneShot(...) using
    MultiSendCallOnly in a single Safe module call to keep
1263+// atomicity while consuming the nonce on failure (no revert;
    emit a failure event).
1264 function dispatchBatch(
1265     address account,
```

Severity

Impact Explanation: [Low] Impacts are limited to execution quality/timing within the manager's signed bounds and the permission's constraints (e.g., slippage bands, LTV caps). There is no direct theft of principal, freezing of funds, or invariant break; recipients and parameters remain as signed.

Likelihood Explanation: [Medium] Exploitation requires observing a valid signature (often via reverted attempts) and timing execution with favorable state (oracle/pool conditions) or a visible in-mempool configure() window. These are plausible but outside full attacker control, with clear profit incentives (MEV).

Exploitation

Exploitation Scenarios:

Scenario 1.

MEV sandwich on a leaked swap signature: A manager signs a SwapPermission-governed swap with amountOutMin set and attempts execution; it reverts (e.g., oracle band not met). The signature is public and nonce unconsumed. An attacker copies the signature, waits until the oracle band and AMM output marginally satisfy the constraints, and submits a bundle: a small frontrun buy, the replayed dispatch, and a backrun sell. The swap executes within the signed bounds but at worse execution due to the sandwich.

Preconditions / Assumptions

- (a). A valid manager signature for a SwapPermission-governed router call becomes public (e.g., due to a prior reverted attempt).
- (b). The permission remains registered and the session active; deadline not expired.
- (c). Oracle freshness and price band conditions can be satisfied later.
- (d). Attacker can submit a bundled transaction to perform a sandwich around the replayed dispatch.

Scenario 2.



Forced borrow timed at an LTV trough: A manager signs a borrow under BorrowPermission near the LTV ceiling. An initial attempt is denied (e.g., due to oracle-reported values), leaving the signature public and nonce unconsumed. The attacker monitors the oracles and replays the signature during a brief favorable window (fresh feeds) so the LTV check passes; the borrow executes to the account, increasing position risk at an attacker-chosen time.

Preconditions / Assumptions

- (a). A valid manager signature for a BorrowPermission-governed borrow becomes public (e.g., prior revert/denial).
- (b). Permission remains registered and the session active; deadline not expired.
- (c). Both collateral and borrow oracle feeds will report fresh, transiently favorable values.
- (d). Attacker monitors and replays during that favorable window.

Scenario 3.

Front-running an in-place configuration tighten: A previously leaked, valid manager signature remains unconsumed. The permissionSigner submits an in-place configure() to tighten rules (not replacePermission), which does not epoch-bump nonces. The attacker front-runs this configure transaction by replaying the old signature before the tighten lands, executing one last action under the old config.

Preconditions / Assumptions

- (a). A valid, previously leaked manager signature remains unconsumed and within deadline.
- (b). The permissionSigner submits an in-place configure() (not replacePermission) that does not epoch-bump nonces.
- (c). The configure transaction is visible in the mempool and can be front-run.
- (d). Permission remains registered; deadline not expired.

Vulnerability #15

Missing feeAsset in CREATE2 salt binding in SailKernel.createAccount causes unintended fee-asset configuration and fee-collection DoS in shared-caller flows

Severity: LOW

Likelihood: LOW

Impact: MEDIUM

Status: Acknowledged

Accepted by design. feeAsset is deliberately excluded from the CREATE2 salt to preserve same-address portability across chains; including it would change every account address. A squatter gains no authority (the salt binds the principals + initializer). Note: after the PR #57 fee-asset binding, recovery from an unintended fee-asset is no longer a same-instance re-point – that reverts FeePolicyAssetMismatch – so recovery requires pointing at a different trusted policy instance. The doc claim that this was "self-curable in a single transaction" on the same instance was corrected in PR #67.

SailKernel.createAccount binds the CREATE2 salt to saltNonce, msg.sender, permissionSigner, manager, feePolicy, and keccak256(safeInitializer), but not to feeAsset, while _registerAccount persists feeAsset. In shared-caller onboarding (same msg.sender for many users), a frontrunner can replay the victim's call with the same salt-bound fields but a different feeAsset, registering first at the identical predicted Safe address. The Safe is owned by the intended owners but configured with an attacker-chosen feeAsset, enabling fee collection in an unintended asset or temporarily DoSing fee collection until the permissionSigner corrects feeAsset via setFeePolicy.

In SailKernel.createAccount the bound salt for CREATE2 address derivation is computed from saltNonce, msg.sender, permissionSigner, manager, feePolicy and the initializer hash, but excludes feeAsset. After deploying/adopting the predicted Safe, _registerAccount stores feeAsset in configs. In a shared-caller flow (e.g., a router/forwarder used by multiple users), an attacker who sees a pending createAccount transaction can invoke the same caller with the same safeInitializer, saltNonce, permissionSigner, manager, and feePolicy but a different feeAsset. Because feeAsset is not part of the salt, the predicted address is identical; the attacker's transaction registers first, persisting the attacker-chosen feeAsset. The victim's later createAccount reverts with AccountAlreadyRegistered, leaving the Safe owned by the intended owners but configured with the wrong feeAsset. collectFees enforces feeToken == cfg.feeAsset, so this misconfiguration can (a) allow fees to be collected in an unintended asset (e.g., ETH) or (b) cause repeated FeeTransferFailed if the attacker-selected feeAsset reverts on transfer, producing a temporary DoS on fee collection. The permissionSigner can remediate by calling setFeePolicy to reset feeAsset. EOA-direct onboarding

(unique msg.sender per user) and forwarders that cryptographically bind the full payload (including feeAsset) prevent this, but the core omission remains exploitable in shared-caller contexts.

file: contracts/core/SailKernel.sol

```
607     if (setupTarget != address(0) && !governance.trustedModuleSetup(setupTarget)) revert UntrustedModuleSetup(setupTarget);
608
609 -   uint256 boundSalt = uint256(keccak256(abi.encode(saltNonce,
610     msg.sender, permissionSigner, manager, feePolicy)));
611     // Predict the proxy address with the same CREATE2 formula
612     SafeProxyFactory uses, so a
```

```
607     if (setupTarget != address(0) && !governance.trustedModuleSetup(setupTarget)) revert UntrustedModuleSetup(setupTarget);
608
609 +   uint256 boundSalt = uint256(keccak256(abi.encode(saltNonce,
610     msg.sender, permissionSigner, manager, feePolicy, feeAsset)));
611     // Predict the proxy address with the same CREATE2 formula
612     SafeProxyFactory uses, so a
```

Severity

Impact Explanation: [Medium] For the DoS scenario, fee collection (a core revenue function) is significantly but temporarily unavailable until corrected by the permission-Signer, fitting Medium impact. The wrong-asset collection scenario is lower impact, but the maximum impact across valid scenarios is Medium.

Likelihood Explanation: [Low] Exploitation requires a specific integration pattern (shared-caller onboarding), public mempool front-running, and a forwarder that does not cryptographically bind all forwarded fields (including feeAsset). These are uncommon but plausible conditions, resulting in Low likelihood.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 – Unintended ETH fee collection: A user onboards via a shared forwarder with feeAsset set to an ERC-20 (e.g., USDC). An attacker front-runs through the same forwarder, keeping safeInitializer, saltNonce, permission-Signer, manager, feePolicy identical but setting feeAsset = ETH. The kernel deploys/adopts the same predicted Safe and registers with feeAsset = ETH. Later, when the manager calls collectFees with feeToken = ETH, the kernel transfers ETH from the Safe per the policy split, resulting in fee collection in the unintended asset until the permissionSigner corrects feeAsset via setFeePolicy.

Preconditions / Assumptions

- (a). Onboarding uses a shared on-chain forwarder/router, so msg.sender at the kernel is identical across multiple users.
- (b). The forwarder does not cryptographically bind all createAccount fields (including feeAsset) under a signature enforced on-chain, allowing a third party to modify feeAsset when relaying; or it permits arbitrary caller-supplied payloads.
- (c). Public mempool visibility allows the attacker to ob-



- serve and front-run the pending createAccount.
- (d). safeInitializer is a valid Safe.setup that enables the kernel module; the provided feePolicy is trusted-allowlisted.
 - (e). The manager or automation attempts to collect fees before the permissionSigner corrects feeAsset.

Scenario 2.

Scenario 2 – Fee-collection DoS: A user onboards via a shared forwarder intending an ERC-20 fee asset. An attacker front-runs through the same forwarder, registering first with feeAsset pointing to a reverting/non-ERC20 address. Subsequent collectFees attempts with this feeToken revert (FeeTransferFailed). Fee collection is unavailable until the permissionSigner updates feeAsset via setFeePolicy.

Preconditions / Assumptions

- (a). Onboarding uses a shared on-chain forwarder/router, so msg.sender at the kernel is identical across multiple users.
- (b). The forwarder does not cryptographically bind all createAccount fields (including feeAsset) under a signature enforced on-chain, allowing a third party to modify feeAsset when relaying; or it permits arbitrary caller-supplied payloads.
- (c). Public mempool visibility allows the attacker to observe and front-run the pending createAccount.
- (d). safeInitializer is a valid Safe.setup that enables the kernel module; the provided feePolicy is trusted-allowlisted.
- (e). The attacker-selected feeAsset address reverts on transfer or is a non-ERC20, causing _safeTransferERC20 to fail.
- (f). The manager attempts fee collection before the permissionSigner corrects feeAsset.

Vulnerability #16

Floor rounding in fee split with manager-chosen grossFee in SailKernel.collectFees/StandardFeePolicy causes protocol/distributor underpayment and burned headroom

Severity: INFORMATIONAL

Likelihood: HIGH

Impact: INFORMATIONAL

Status: Acknowledged

Accepted. The shortfall accrues to Sail's own treasury/distributor (protocol cut is 0 at launch → ~zero impact), is bounded to 1–2 base units, and cannot compound intraday (MIN_COLLECTION_INTERVAL = 1 day). grossFee is manager-chosen up to maxFee by design; forcing grossFee == maxFee would remove the intended ability to under-collect.

In collectFees, protocolCut and distributorCut use floor division from a manager-chosen grossFee (only constrained by grossFee ≤ maxFee). This creates rounding plateaus where different grossFee values yield the same managerTake but smaller protocol/distributor payouts. StandardFeePolicy.recordCollection then advances timestamp/HWM, permanently burning any unused headroom. Impact is bounded to 1–2 base units per collection and is an intended design tradeoff.

SailKernel.collectFees splits grossFee into: protocolCut = floor(grossFee × protocolBps / 10,000), remainder = grossFee – protocolCut, distributorCut = floor(remainder × distributorBps / 10,000) (0 if distributor = 0), and managerTake = remainder – distributorCut. Because Math.mulDiv floors, certain adjacent grossFee integers form plateaus with an unchanged managerTake while protocol and/or distributor receive less (by 1–2 minimal units across the plateau span). The kernel only enforces grossFee ≤ maxFee (from IFeePolicy.computeFee) and does not require grossFee = maxFee, so the manager can intentionally choose lower plateau points without reducing their own take. StandardFeePolicy.recordCollection is called before transfers and advances lastCollectionTimestamp and the high-water mark (to max(HWM, currentNav)), so any uncollected headroom is irreversibly lost; MIN_COLLECTION_INTERVAL = 1 day prevents intraday micro-splitting. Authorized callers are only the manager or the Safe, feeToken must match the configured feeAsset, and transfers follow CEI ordering with enforced ceilings. The effect is bounded to base-unit granularity per collection and is consistent with the intended flexibility to collect less than the ceiling and with integer floor arithmetic.

file: contracts/core/SailKernel.sol



```

1504 address feePolicy = cfg.feePolicy;
1505 if (feePolicy == address(0)) revert FeePolicyNotSet();
1506- if (grossFee == 0) revert ZeroFee();
1507 if (feeToken != cfg.feeAsset) revert FeeTokenMismatch(fee-
    Token, cfg.feeAsset);

...

1513 (uint256 maxFee, address distributor, uint256 distribu-
    torBps) =
1514 IFeePolicy(feePolicy).computeFee(account, currentNav);
1515- if (grossFee > maxFee) revert FeeTooLarge(grossFee, maxFee);

1516 if (distributorBps > 10_000) revert DistributorBp-
    sTooLarge(distributorBps);

```

```

1504 address feePolicy = cfg.feePolicy;
1505 if (feePolicy == address(0)) revert FeePolicyNotSet();

1506 if (feeToken != cfg.feeAsset) revert FeeTokenMismatch(fee-
    Token, cfg.feeAsset);

...

1512 (uint256 maxFee, address distributor, uint256 distribu-
    torBps) =
1513 IFeePolicy(feePolicy).computeFee(account, currentNav);
1514+ // Enforce collection of the full authorised amount to
    avoid underpayment plateaus.
1515+ grossFee = maxFee;
1516+ if (grossFee == 0) revert ZeroFee();
1517 if (distributorBps > 10_000) revert DistributorBp-
    sTooLarge(distributorBps);

```

Severity

Impact Explanation: [Informational] Effects are bounded to baseunit rounding differences (typically 1–2 minimal units per collection), cannot be compounded intraday due to a 1-day minimum interval, and reflect allowed undercollection rather than a constraint bypass.

Likelihood Explanation: [High] Once an account is active and eligible for collection, the manager can freely choose any $\text{grossFee} \leq \text{maxFee}$; no special constraints or uncommon conditions are required.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (no distributor): With protocol cut 25%, the manager chooses $\text{grossFee} = 3$ instead of 4. Both $\text{yield managerTake} = 3$, but protocolCut is 0 vs 1 respectively. recordCollection advances timestamp/HWM and the 1-unit protocol shortfall cannot be reclaimed.

Preconditions / Assumptions

- (a). Account is registered and sessionActive is true
- (b). $\text{configs}[\text{account}].\text{feePolicy}$ is StandardFeePolicy and trusted; feeToken equals $\text{configs}[\text{account}].\text{feeAsset}$
- (c). $\text{StandardFeePolicy.hwmSeeded}[\text{account}]$ is true and at least 1 day has elapsed since last collection
- (d). $\text{governance.currentProtocolCutBps}()$ is set (e.g., 25%)
- (e). No distributor configured ($\text{distributor} == \text{address}(0)$ or $\text{distributorBps} == 0$)
- (f). $\text{IFeePolicy.computeFee}(\text{account}, \text{currentNav})$ returns $\text{maxFee} \geq 4$ minimal units
- (g). Caller is $\text{configs}[\text{account}].\text{manager}$ or the Safe account
- (h). Treasury and feeRecipient are valid and payable if ETH
- (i). ERC20 behavior matches assumptions (no feeontransfer , etc.)

Scenario 2.

Scenario 2 (with distributor 50% of remainder): For gross-



Fee {3,4,5} under protocol cut 25% and distributorBps 50%, managerTake stays 2 while protocol+distributor increase across the set. Picking 3 (lowest plateau point) instead of 5 starves protocol by 1 and distributor by 1; recordCollection advances state and burns the 2-unit headroom.

Preconditions / Assumptions

- (a). Account is registered and sessionActive is true
- (b). configs[account].feePolicy is StandardFeePolicy and trusted; feeToken equals configs[account].feeAsset
- (c). StandardFeePolicy.hwmSeeded[account] is true and at least 1 day has elapsed since last collection
- (d). governance.currentProtocolCutBps() is set (e.g., 25%)
- (e). StandardFeePolicy.distributor is set and distributorBps = 50%
- (f). IFeePolicy.computeFee(account, currentNav) returns $\text{maxFee} \geq 5$ minimal units
- (g). Caller is configs[account].manager or the Safe account
- (h). Treasury, distributor, and feeRecipient are valid and payable if ETH
- (i). ERC20 behavior matches assumptions (no feeontransfer, etc.)

Scenario 3.

Scenario 3 (0decimal fee token): Same as Scenario 2 but feeAsset has 0 decimals. Choosing grossFee = 3 vs 5 with the same managerTake causes a 2token shortfall to protocol+distributor in that period; state advance burns the headroom. Magnitude depends on the configured feeAsset decimals.

Preconditions / Assumptions

- (a). All preconditions from Scenario 2
- (b). Configured feeAsset is an ERC20 with 0 decimals (deployment choice)
- (c). IFeePolicy.computeFee(account, currentNav) returns $\text{maxFee} \geq 5$ whole tokens

Vulnerability #17

Permissionless configure() and pre-registration sequencing in MandateFactory enable mempool replay to DoS attach flows

Severity: INFORMATIONAL

Likelihood: LOW

Impact: LOW

Status: Acknowledged

Accepted design tradeoff. MandateFactory is the untrusted UX orchestrator (whitepaper §4.6) and holds no protocol privilege; every inner call is independently EIP-712 authenticated, so an arbitrary submitter cannot change what is configured – only when a pre-signed bundle lands. A replayed configureSig makes the bundle revert benignly (no permission activated) and recovery needs no new signature (submit SailKernel.register* directly with the existing kernel-Sig). Same family as #14; operational mitigation in Sailor.

A mempool observer can copy a valid configure signature from a pending MandateFactory.attach/attachBatch/replace call and submit it directly to the template before the factory tx lands. This advances the template's config nonce, causing the factory's _batchConfigure/configure step to revert and the whole factory transaction to fail, preventing the intended one-tx configure→register UX. No funds or kernel invariants are affected; operators can immediately complete registration directly on SailKernel using the already-signed kernel* signature.

MandateFactory.attachBatch executes template configuration first via _batchConfigure and only then calls SailKernel.registerPermissions. Each template's IConfigurablePermission.configure is permissionless to call given a valid EIP-712 signature and checks a per-template, per-account nonce (configNonces). If a pending factory transaction reveals any configureSig in the public mempool, an external observer can call that template's configure(account, params, deadline, sig) before the factory tx is included. This succeeds, consumes the template's config nonce, and applies the signed params. When the original attachBatch later runs, _batchConfigure reuses the same signature and reverts on nonce mismatch, so the entire factory call fails and kernel.registerPermissions is never reached. The same grief applies to MandateFactory.attach (single) and replace (configures newTemplate before replacing). The resulting state is benign (some templates may be configured, but none are registered, so no permission is active). Recovery is straightforward without new signatures for the kernel step: the operator can call SailKernel.registerPermissions (or registerPermission/replacePermission) directly using the same pre-signed kernel*Sig from the failed factory call. This is a low-impact, pure grief/DoS of a UX-bundling function, preventable with private mempool submission.


```
100 ) external payable nonReentrant {
101     uint256 preBalance = address(this).balance - msg.value;

102     IConfigurablePermission(template).configure(account,
params, configureDeadline, configureSig);
103     kernel.registerPermission{value: msg.value}(account,
template, kernelDeadline, kernelSig);

...

126     uint256 preBalance = address(this).balance - msg.value;
127

128     _batchConfigure(account, templates, params, configureDead-
lines, configureSigs);
129     kernel.registerPermissions{value: msg.value}(account,
templates, kernelDeadline, kernelBatchSig);

...

139     bytes[] calldata configureSigs
140 ) private {

141     for (uint256 i; i < templates.length; i++) {
142         IConfigurablePermission(templates[i]).configure(
...

176 ) external payable nonReentrant {
177     uint256 preBalance = address(this).balance - msg.value;

178     IConfigurablePermission(newTemplate).configure(
179     account, newParams, configureDeadline, configureSig
```

```
100 ) external payable nonReentrant {
101     uint256 preBalance = address(this).balance - msg.value;
102 +
```

```

    // WARNING: See _batchConfigure mitigation note for V2
    nonce-aware flow to tolerate pre-consumed configure signatures.
103   IConfigurablePermission(template).configure(account,
    params, configureDeadline, configureSig);
104   kernel.registerPermission{value: msg.value}(account,
    template, kernelDeadline, kernelSig);

    ...

127   uint256 preBalance = address(this).balance - msg.value;
128
129 +// WARNING: Public-mempool replay of configure() can revert
    this batch; implement V2 with expected nonces
130 +// and nonce-aware configure-or-skip to tolerate
    pre-consumed signatures. See _batchConfigure note below.
131   _batchConfigure(account, templates, params, configureDead-
    lines, configureSigs);
132   kernel.registerPermissions{value: msg.value}(account,
    templates, kernelDeadline, kernelBatchSig);

    ...

142   bytes[] calldata configureSigs
143   ) private {
144 +// NOTE: Mitigation guidance:
145 +// Add V2 entrypoints that accept expected config nonces
    per template and implement a nonce-aware
146 +// "configure-or-skip-if-already-applied" flow using
    pre-read nonce checks and try/catch with
147 +// post-revert nonce recheck. Old entrypoints remain for
    backward compatibility; tooling should migrate.
148   for (uint256 i; i < templates.length; i++) {
149   IConfigurablePermission(templates[i]).configure(

    ...

183   ) external payable nonReentrant {
184   uint256 preBalance = address(this).balance - msg.value;
185 +// WARNING: See _batchConfigure mitigation note for V2
    nonce-aware flow to tolerate pre-consumed configure signatures.
186   IConfigurablePermission(newTemplate).configure(
187   account, newParams, configureDeadline, configureSig

```

Severity

Impact Explanation: [Low] Impact is limited to minor UX friction and temporary DoS of the factory's one-tx bundling flow (a revert and the need for a follow-up direct kernel call). No funds or kernel invariants are affected, and there is an immediate, simple workaround without new signatures for the kernel step.

Likelihood Explanation: [Low] The attack is pure grieving with no profit; the attacker must spend gas to cause reverts. It requires the operator to use a public mempool; using private relays/builders prevents it.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1 (attachBatch): Operator broadcasts `MandateFactory.attachBatch` with `N` configure signatures and a kernel batch signature via a public mempool. An observer copies one `configureSig` for `templates[k]` and calls `templates[k].configure(account, params[k], deadline[k], sig[k])` first, advancing the template's config nonce. When the original `attachBatch` runs, `_batchConfigure` reverts on that reused signature, so the whole transaction fails before `kernel.registerPermissions`. Operator loses gas and must submit `SailKernel.registerPermissions` directly using the already-signed `kernelBatchSig` to complete registration, then (re)configure any remaining templates.

Preconditions / Assumptions

- (a). The `Safe` account is already registered in `SailKernel` (`registered[account] == true`).
- (b). Operator submits `MandateFactory.attachBatch` via a public mempool, revealing `configureSigs` and `kernelBatchSig`.
- (c). `configure` signatures are valid and not expired for each template.
- (d). Attacker can see mempool transactions and is willing to pay gas to submit `configure`.



Scenario 2.

Scenario 2 (attach, single): Operator broadcasts `MandateFactory.attach` with `configureSig` and `kernelSig`. An observer pre-calls `template.configure` with the revealed signature, consuming the nonce. The factory's `configure` step reverts, so `attach` fails before `SailKernel.registerPermission`. Operator then calls `SailKernel.registerPermission` directly with the same `kernelSig` to complete registration.

Preconditions / Assumptions

- (a). The Safe account is already registered in `SailKernel`.
- (b). Operator submits `MandateFactory.attach` via a public mempool, revealing `configureSig` and `kernelSig`.
- (c). `configure` signature is valid and not expired.
- (d). Attacker can see mempool transactions and is willing to pay gas to submit `configure`.

Scenario 3.

Scenario 3 (replace): Operator broadcasts `MandateFactory.replace` to configure `newTemplate` and then replace `oldTemplate` with `newTemplate`. An observer pre-calls `newTemplate.configure` with the revealed signature, consuming the nonce. The factory's `configure` step reverts, and `replace` fails before `SailKernel.replacePermission`. Operator calls `SailKernel.replacePermission` directly with the same `kernelReplaceSig` to complete the swap.

Preconditions / Assumptions

- (a). The Safe account is already registered; `oldTemplate` is registered on the account.
- (b). Operator submits `MandateFactory.replace` via a public mempool, revealing `newTemplate's configureSig` and `kernelReplaceSig`.
- (c). `newTemplate's configure` signature is valid and not expired.
- (d). Attacker can see mempool transactions and is willing to pay gas to submit `configure`.



Vulnerability #18

Same-caller CREATE2 salt namespace and no idempotent adoption in MandateFactory.deployAndAttach cause onboarding DoS/griefing

Severity: INFORMATIONAL

Likelihood: LOW

Impact: INFORMATIONAL

Status: Resolved

Fixed in PR #58. The clone CREATE2 salt now binds the account in addition to msg.sender, isolating distinct accounts under a shared caller (relayer/aggregator), mirroring the kernel's bound-salt doctrine. A same-account front-run via a shared relayer remains out of scope (would require a signed deployAndAttachV2) and is flagged.

MandateFactory.deployAndAttach namespaces the CREATE2 salt only by msg.sender and lacks an adopt-existing-clone path. In shared-caller (relayer/aggregator) setups, different users share one deterministic clone namespace, enabling an attacker to front-run and deploy a clone at a victim's predicted address. This forces the victim to re-sign with a new salt/address, and in a stronger variant lets the attacker consume the victim's permission registration signature to register a wrongly-initialized permission on the victim's account. Impact is operational/UX friction; no direct fund loss.

Root cause: MandateFactory.deployAndAttach derives namespacedSalt = keccak256(abi.encode(msg.sender, salt)) and deploys via Clones.cloneDeterministic without first checking if code already exists at the predicted address. There is no idempotent "adopt existing clone" fallback. In shared-caller environments (e.g., public relayers/aggregators) multiple users share the same msg.sender, therefore the same CREATE2 namespace for a given (impl, salt). An attacker can front-run with the same (impl, salt) through the same caller to occupy the victim's predicted clone address first. SailKernel.registerPermission authorizes by an EIP-712 signature from the account's permissionSigner and only checks that the permission address has code; it does not verify that a standalone clone's initialize() was performed for that account. Consequently, an attacker who sees the victim's registration signature in the mempool can register the attacker-initialized clone on the victim's account, leaving a non-functional/unsafe permission that the victim must revoke before re-onboarding. The attacker bears the registration fee and gas, so this is a griefing/DoS vector rather than a profit path. Built-in templates in this repo use ConfigurablePermission (shared instances) and do not rely on deployAndAttach; however, MandateFactory explicitly supports third-party standalone initialize-based templates, where this risk is applicable. Operational mitigations include avoiding shared callers for deployAndAttach, using private relays/bundles, and keeping salts secret until execution.

file: contracts/factory/MandateFactory.sol

```
191 // - impl: the logic contract address (from
    deployments/<chainId>/templates.standalone.json)
192 // - salt: caller-chosen entropy; the factory namespaces
    it internally as
193 -// keccak256(abi.encode(msg.sender, salt)) to
    prevent cross-caller
194 -// salt squatting. Use keccak256(abi.encode(account,
    impl, perAccountNonce))
195 -// as the raw salt to give each account its
    own address space.
196 // - initData: ABI-encoded initialize(...) call (selector
    + args)
197 // - kernelSig: permission-signer signature for kernel.reg-
    isterPermission
...
209 /// that implement `initialized()` incorrectly can
    still pass this check
210 /// while remaining misconfigured. Verify `initData`
    correctness off-chain.
...
211 function deployAndAttach(
212     address account,
...
222     uint256 preBalance = address(this).balance - msg.value;
223
224 - bytes32 namespacedSalt = keccak256(abi.encode(msg.sender,
    salt));
225     clone = Clones.cloneDeterministic(impl, namespacedSalt);
...
242 /// @notice Predict the address of a clone before it is
    deployed.
243 /// Must be called from the same EOA that will call
    `deployAndAttach`,
244 - /// because the factory namespaces the salt by
    msg.sender.
```



```

245 - function predictCloneAddress(address impl, bytes32 salt)
    external view returns (address) {
246 - bytes32 namespacedSalt = keccak256(abi.encode(msg.sender,
    salt));
247     return Clones.predictDeterministicAddress(impl, namespaced-
    Salt, address(this));
248 }

```

```

191 // - impl:         the logic contract address (from
    deployments/<chainId>/templates.standalone.json)
192 // - salt:         caller-chosen entropy; the factory namespaces
    it internally as
193 +//               keccak256(abi.encode(msg.sender, account,
    salt)) to prevent cross-caller
194 +//               and within-caller (shared relayer) salt
    squatting.

195 // - initData:     ABI-encoded initialize(...) call (selector
    + args)
196 // - kernelSig:     permission-signer signature for kernel.reg-
    isterPermission

...

208 ///             that implement `initialized()` incorrectly can
    still pass this check
209 ///             while remaining misconfigured. Verify `initData`
    correctness off-chain.
210 +// NOTE: To harden against param-tampering front-runs,
    introduce deployAndAttachV2
211 +// that verifies a permissionSigner EIP-712 signature binding
    initData to (account, impl, salt).
212     function deployAndAttach(
213         address account,

...

223     uint256 preBalance = address(this).balance - msg.value;
224
225 + bytes32 namespacedSalt = keccak256(abi.encode(msg.sender,
    account, salt));
226     clone = Clones.cloneDeterministic(impl, namespacedSalt);

...

```

```
243  /// @notice Predict the address of a clone before it is
    deployed.
244  ///          Must be called from the same EOA that will call
    `deployAndAttach`,
245 +///          because the factory namespaces the salt by msg.sender
    and account.
246 + function predictCloneAddress(address impl, address account,
    bytes32 salt) external view returns (address) {
247 + bytes32 namespacedSalt = keccak256(abi.encode(msg.sender,
    account, salt));
248     return Clones.predictDeterministicAddress(impl, namespaced-
    Salt, address(this));
249 }
```


Severity

Impact Explanation: [Informational] No direct loss of principal or fees; protocol functionality remains intact. Impacts are operational/UX friction: failed onboarding attempts, forced re-signing with new salts/addresses, and in one variant a cleanup revoke that bumps nonces for a single account.

Likelihood Explanation: [Low] Exploitation requires combined preconditions: a shared-caller setup, public mempool visibility, use of `deployAndAttach` for standalone clones, and an attacker willing to spend registration fees and gas without direct profit. These constraints reduce practical frequency and incentives.

Exploitation

Exploitation Scenarios:

Scenario 1.

Scenario 1: Attacker front-runs via the same shared caller to `deploy+initialize` the clone for their own account using the victim's (`impl`, `salt`), then immediately submits `SailKernel.registerPermission` for the victim's account using the victim's pre-signed `kernelSig` observed in the mempool. The victim's `deployAndAttach` later reverts on `CREATE2`; their signer nonce is consumed; a wrongly-initialized permission sits on their account and must be revoked before re-onboarding with a new salt/address.

Preconditions / Assumptions

- (a). A public shared-caller (relayer/aggregator) is used to call `MandateFactory.deployAndAttach`, so `msg.sender` is identical across users.
- (b). The victim's Safe account is already registered with `SailKernel`.
- (c). The permission is a standalone, `initialize()`-based clone using `CloneInitializable` and binds state to an account at initialization.
- (d). The victim precomputed `predictCloneAddress(impl, salt)` using the shared caller's address and obtained a valid `permissionSigner` EIP-712 signature (`kernelSig`) over



- (accountVictim, predictedClone, nonce, deadline).
- (e). The victim's transaction (including salt and kernelSig) is broadcast in the public mempool.
- (f). The attacker pays the protocol registration fee and gas.

Scenario 2.

Scenario 2: Attacker front-runs via the same shared caller using the victim's (impl, salt) to deploy+initialize+register the clone for the attacker's account. The victim's deployAndAttach reverts on CREATE2, forcing them to re-sign with a new salt/address. No funds are lost.

Preconditions / Assumptions

- (a). A public shared-caller (relayer/aggregator) is used to call MandateFactory.deployAndAttach, so msg.sender is identical across users.
- (b). The victim's transaction reveals (impl, salt) in the public mempool prior to inclusion.
- (c). The attacker pays the protocol registration fee and gas.
- (d). The victim relies on a precomputed predicted address for onboarding.

Scenario 3.

Scenario 3: Attacker pre-squats common/predictable salts (e.g., 0x00, 0x01) through the same shared caller by deploying+registering clones for their own account. Later users who attempt deployAndAttach with those salts in that shared namespace revert on CREATE2 and must pick new salts and re-sign.

Preconditions / Assumptions

- (a). A public shared-caller (relayer/aggregator) is used to call MandateFactory.deployAndAttach.
- (b). Some users or tooling use predictable/reused salts within that shared namespace.
- (c). The attacker is willing to pay the registration fee repeatedly to pre-squat those salts.

Warning #1

msg.sender-based Safe authorization in SailKernel reachable via Safe fallback (fallbackHandler = SailKernel) causes unauthorized registration/manager-rotation/fee outflows

Severity: MEDIUM

Likelihood: LOW

Impact: HIGH

Status: Resolved

Fixed in PR #53. The warning is subsumed by the owner-signature requirement added to registerAccount: registration now requires a Safe-owner EIP-712 signature (verified via Safe-core checkSignatures) and verifies the trusted singleton ahead of any other ISafe call, so the fallbackHandler-as-kernel calldata path no longer provides an unauthorized registration route. setManager/collectFees retain exact-length calldata guards as named constants.

SailKernel functions that authorize solely by msg.sender == account (the Safe) – setManager, registerAccount, and collectFees – can be invoked by any external user through the Safe's fallback when the Safe's fallback handler is set to the SailKernel contract. Gnosis Safe fallback forwards unknown calls to the handler via CALL, making msg.sender the Safe and appending 20 bytes to calldata (which Solidity tolerates). This enables unauthorized account registration with attacker principals (leading to theft), unauthorized manager rotation with mandate clearing (DoS/hijack), and unauthorized fee collections (fund outflows).

Gnosis Safe v1.4.1's FallbackManager forwards unknown calls to the configured fallback handler by CALL, setting msg.sender to the Safe and appending the original caller's address to calldata. Solidity external function decoding accepts trailing bytes, so SailKernel's external functions decode normally. SailKernel's setManager and registerAccount rely on msg.sender == account (the Safe) for authorization, and collectFees allows msg.sender == manager OR msg.sender == account. If a Safe configures its fallback handler to the SailKernel address (which is permitted by the system: createAccount only constrain Safe.setup delegatecall 'to', not fallbackHandler), any EOA can call the Safe directly with calldata for these functions; the Safe's fallback relays to the kernel, satisfying the msg.sender == account check. Consequences:

Unauthorized registerAccount lets the attacker set themselves as permission-Signer and manager, then register permissive permissions and drain funds via dispatch.

Unauthorized setManager rotates the manager to the attacker, clears all permissions, and bumps nonce epochs, causing persistent DoS/hijack until fallback is fixed.

Unauthorized collectFees forces fee outflows (to treasury/distributor/feeRecipient) using attacker-chosen currentNav and grossFee within policy bounds. These

are direct losses to the account owners, repeatable each collection interval. Safe itself does not prevent setting the handler to an arbitrary contract (only forbids handler == self). SailKernel does not detect fallback-sourced calls (e.g., via strict calldata length), and does not constrain fallbackHandler in createAccount/registerAccount. Thus, this is an authorization bypass under an admitted configuration. Victims are SMAs/Safe owners whose Safe fallback handler is set to SailKernel.

file: contracts/core/SailKernel.sol

```
455     error ProtocolPaused();
456
457     /// @dev Thrown when `createAccount` deploys a Safe that does
458     not have this kernel enabled as a module.
459     error ModuleNotEnabled();
460
461     ...
462
463     /// @param feeAsset Canonical fee settlement token;
464     address(0) = native ETH.
465     function registerAccount(address permissionSigner, address
466     manager, address feePolicy, address feeAsset) external {
467
468         bytes32 codehash;
469         assembly { codehash := extcodehash(caller()) }
470
471         ...
472
473         /// different from the current manager.
474         function setManager(address newManager) external nonReentrant
475         {
476
477             address account = msg.sender;
478             _requireRegistered(account);
479
480             ...
481
482             address feeToken
483             ) external nonReentrant whenNotPaused {
484
485                 _requireRegistered(account);
486                 AccountConfig storage cfg = configs[account];
```

```

455     error ProtocolPaused();
456
457 +/// @dev Thrown when a Safe-authorized function is called with
    unexpected calldata length (e.g., via Safe fallback).
458 + error UnexpectedCalldataLength();
459     /// @dev Thrown when `createAccount` deploys a Safe that does
    not have this kernel enabled as a module.
460     error ModuleNotEnabled();

...

654     /// @param feeAsset          Canonical fee settlement token;
    address(0) = native ETH.
655     function registerAccount(address permissionSigner, address
    manager, address feePolicy, address feeAsset) external {
656 + if (msg.data.length != 132) revert UnexpectedCalldataL-
    ength();
657     bytes32 codehash;
658     assembly { codehash := extcodehash(caller()) }

...

710     ///                different from the current manager.
711     function setManager(address newManager) external nonReentrant
    {
712 + if (msg.data.length != 36) revert UnexpectedCalldataL-
    ength();
713     address account = msg.sender;
714     _requireRegistered(account);

...

1496     address feeToken
1497 ) external nonReentrant whenNotPaused {
1498+ if (msg.data.length != 132) revert UnexpectedCalldataL-
    ength();
1499     _requireRegistered(account);
1500     AccountConfig storage cfg = configs[account];

```

Severity

Impact Explanation: [High] Unauthorized registerAccount enables attacker-controlled permissionSigner/manager and permissive mandates to steal principal; unauthorized collectFees causes direct, material fund outflows to fee recipients; unauthorized setManager causes durable DoS/hijack.

Likelihood Explanation: [Low] All scenarios require a privileged and uncommon configuration (Safe fallback handler set to SailKernel) that attackers cannot induce, plus in scenario 1 an enabled-but-unregistered timing window; these are notable operator-driven constraints.

Exploitation

Exploitation Scenarios:

Scenario 1.

Unauthorized registerAccount takeover and theft: A genuine Safe proxy has the SailKernel module enabled but is not yet registered; its fallback handler is set to SailKernel. An attacker sends a transaction to the Safe with calldata for SailKernel.registerAccount(attackerPermissionSigner, attackerManager, feePolicy, feeAsset). Safe forwards to the kernel via fallback; inside SailKernel, msg.sender == the Safe. Kernel verifies Safe codehash and module enabled, then registers attacker principals. The attacker (as permissionSigner) registers permissive permissions and (as manager) dispatches arbitrary fund-moving calls via the Safe, draining assets.

Preconditions / Assumptions

- (a). Safe is a genuine Safe proxy (trusted codehash).
- (b). SailKernel module is enabled on the Safe.
- (c). The Safe is not yet registered with SailKernel.
- (d). Safe's fallback handler is set to the SailKernel contract address.
- (e). Attacker can send a transaction to the Safe.

Scenario 2.

Unauthorized setManager rotation and durable DoS/hijack:



A registered account's fallback handler equals SailKernel. An attacker calls the Safe with calldata for SailKernel.setManager(attackerManager). Safe forwards to the kernel; msg.sender == the Safe. Kernel rotates the manager to the attacker, clears all permissions, and bumps nonce epochs, rendering the account unable to dispatch. The attacker can re-rotate at will until owners change the fallback handler; if owners later re-register permissions, they become usable by the attacker-manager.

Preconditions / Assumptions

- (a). Account is registered with SailKernel.
- (b). SailKernel module is enabled on the Safe.
- (c). Safe's fallback handler is set to the SailKernel contract address.
- (d). Attacker can send a transaction to the Safe.

Scenario 3.

Unauthorized forced fee collection (fund outflows): An account is registered, sessionActive true, a fee policy is set and initialized (e.g., StandardFeePolicy with HWM seeded; MIN_COLLECTION_INTERVAL elapsed), and fallback handler is SailKernel. An attacker calls the Safe with calldata for SailKernel.collectFees(account, grossFee, currentNav, feeToken). Safe forwards to the kernel; msg.sender == the Safe satisfies authorization. Kernel enforces grossFee <= maxFee (computed from attacker-chosen currentNav), records the collection, and transfers protocolCut, distributorCut, and managerTake to treasury, distributor, and feeRecipient, respectively, depleting the account's assets without the manager's consent.

Preconditions / Assumptions

- (a). Account is registered; sessionActive is true.
- (b). A fee policy is set and permits collection (e.g., StandardFeePolicy with HWM seeded and MIN_COLLECTION_INTERVAL elapsed).
- (c). feeAsset matches the intended fee token.
- (d). Safe's fallback handler is set to the SailKernel contract address.
- (e). Attacker can send a transaction to the Safe.

Warning #2

Address-only trustedModuleSetup allowlist in SailKernel.createAccount permits unsafe Safe.setup delegatecall causing takeover of newly created Safes

Severity: MEDIUM

Likelihood: LOW

Impact: HIGH

Status: Resolved

Fixed in PR #64. createAccount now pins the Safe.setup delegatecall target's runtime codehash, in addition to the existing address allowlist, against the immutable SafeModuleEnabler (EXPECTED_SETUP_CODEHASH, captured at kernel construction; reverts UntrustedModuleSetupCodehash on mismatch). The previously operational-only "allowlist must be an immutable helper" rule is now enforced in code – an allowlisted address whose bytecode differs from the audited enabler is rejected.

SailKernel.createAccount validates the Safe.setup delegatecall target only by address against governance.trustedModuleSetup, without binding code identity or immutability. If an allowlisted helper is upgradable/metamorphic and later upgraded to malicious code, createAccount will still delegatecall it during Safe.setup. The helper can modify Safe storage (owners/threshold, guard/fallback, or even the proxy's singleton pointer) while also enabling the kernel module so post-checks pass, resulting in compromised accounts at creation.

During account creation, SailKernel.createAccount parses the Safe.setup "to" (delegatecall target) from the initializer and only checks that it is either address(0) or present in governance.trustedModuleSetup. No codehash or immutability is enforced for the trusted helper address. Safe.setup then calls ModuleManager.setupModules(to, data), which delegatecalls into the helper in the Safe proxy's storage context. Malicious code at the trusted address can: (1) call onlySelf-guarded Safe functions to enable the kernel module and then change owners/threshold, set a guard or fallback handler, or enable additional modules; and/or (2) directly write storage, including slot 0 (SafeProxy.singleton) to point to an attacker-controlled singleton. After setup, the kernel only checks that (a) the kernel module is enabled and (b) the proxy extcodehash matches a trusted SafeProxy runtime codehash. Both still pass. The kernel then registers the account, leaving the newly created Safe compromised while appearing valid. This constitutes an on-chain bypass of the intended trustedModuleSetup defense against unsafe delegatecalls, enabling account takeover and theft for subsequently created accounts using the mutated helper.

file: contracts/core/SailKernel.sol




```

13  import {IERC20}                from "@openzeppelin/contracts/to-
    ken/ERC20/IERC20.sol";
14  import {Math}                  from "@openzeppelin/con-
    tracts/utils/math/Math.sol";

15
16  /// @dev Minimal Safe factory interface – used only in `createAc-
    count`.

...

605  address setupTarget = ad-
    dress(uint160(uint256(bytes32(safeInitializer[68:100]))));
606  // address(0) means no delegatecall (vanilla Safe.setup)
    – always safe, no allowlist check needed.
607 - if (setupTarget != address(0) && !governance.trustedMod-
    uleSetup(setupTarget)) revert UntrustedModuleSetup(setupTarget);

608
609  uint256 boundSalt = uint256(keccak256(abi.encode(saltNonce,
    msg.sender, permissionSigner, manager, feePolicy)));

```

```

13  import {IERC20}                from "@openzeppelin/contracts/to-
    ken/ERC20/IERC20.sol";
14  import {Math}                  from "@openzeppelin/con-
    tracts/utils/math/Math.sol";
15 +import {SafeModuleEnabler}    from "../safe/SafeModuleEn-
    abler.sol";
16
17  /// @dev Minimal Safe factory interface – used only in `createAc-
    count`.

...

606  address setupTarget = ad-
    dress(uint160(uint256(bytes32(safeInitializer[68:100]))));
607  // address(0) means no delegatecall (vanilla Safe.setup)
    – always safe, no allowlist check needed.
608 + if (setupTarget != address(0)) {
609 +

```

```
        if (!governance.trustedModuleSetup(setupTarget)) revert
        UntrustedModuleSetup(setupTarget);
610 + bytes32 _codehash; assembly { _codehash :=
        extcodehash(setupTarget) }
611 + if (_codehash != keccak256(type(SafeModuleEnabler).run-
        timeCode)) revert UntrustedModuleSetup(setupTarget);
612 +}
613
614     uint256 boundSalt = uint256(keccak256(abi.encode(saltNonce,
        msg.sender, permissionSigner, manager, feePolicy)));
```

Severity

Impact Explanation: [High] Account takeover at creation enables direct, material loss of principal funds and full control over the Safe (owners/threshold, modules, guard/fallback, or singleton), satisfying High impact.

Likelihood Explanation: [Low] Exploitation requires that governance has allowlisted an upgradable helper and that the helper's upgrade/admin authority subsequently deploys malicious code at that address (integration/admin compromise). These are special, constrained preconditions outside a random attacker's default control.

Exploitation

Exploitation Scenarios:

Scenario 1.

Owners/threshold hijack at creation: An allowlisted setup helper address H is later upgraded to malicious code M. During Safe.setup, M delegatecalls in the Safe context, enables the kernel module (to satisfy kernel checks) and then replaces the Safe's owners/threshold to make the attacker the sole owner. Kernel post-checks pass and the account is registered, but the attacker now fully controls the Safe and can drain funds.

Preconditions / Assumptions

- (a). Governance previously added address H to governance.trustedModuleSetup (address-only allowlist).
- (b). H is an upgradable/metamorphic helper; the upgrade/admin authority is malicious or compromised and deploys malicious code M at H.
- (c). Victims use H as Safe.setup(to) when calling SailKernel.createAccount (normal integration flow for one-shot module enable).

Scenario 2.

Singleton pointer hijack: The allowlisted helper H is upgraded to code M that, when delegatecalled during Safe.setup, enables the kernel module and then overwrites the SafeProxy's slot 0 singleton pointer to an attacker-con-



trolled singleton S'. Kernel post-checks (module enabled and SafeProxy codehash) still pass; future behavior routes to S', allowing arbitrary subversion and fund theft.

Preconditions / Assumptions

- (a). Governance previously added address H to governance.trustedModuleSetup (address-only allowlist).
- (b). H is an upgradable/metamorphic helper; the upgrade/admin authority is malicious or compromised and deploys malicious code M at H.
- (c). Victims use H as Safe.setup(to) when calling SailKernel.createAccount.
- (d). Attacker has deployed a compatible attacker-controlled singleton S' to hijack the SafeProxy.masterCopy.

Scenario 3.

Backdoor module enabled: The allowlisted helper H is upgraded to code M that, during Safe.setup, enables both the kernel and an attacker-controlled module A. Kernel post-checks pass and the account is registered. Later, the attacker invokes A to execute arbitrary module transactions from the Safe and drain funds without owner or kernel participation.

Preconditions / Assumptions

- (a). Governance previously added address H to governance.trustedModuleSetup (address-only allowlist).
- (b). H is an upgradable/metamorphic helper; the upgrade/admin authority is malicious or compromised and deploys malicious code M at H.
- (c). Victims use H as Safe.setup(to) when calling SailKernel.createAccount.
- (d). Attacker has deployed a malicious module A to be enabled during setup.